



Università
degli Studi
di Palermo



Clustering e Classificazione

INTELLIGENZA ARTIFICIALE PER L'INGEGNERIA BIOMEDICA (EX ROBOTICA MEDICA)
a.a. 2025/2026

Prof. Roberto Pirrone



Clustering

- Il clustering è il processo di partizionamento di un data set in modo che ogni partizione (o *cluster*) contenga dati molto simili tra loro che rispetto agli appartenenti ad altre partizioni
- Rappresenta una classe di algoritmi di apprendimento non supervisionati che sono i più utilizzati nel *data mining*
 - Data summarization
 - Customer segmentation, collaborative filtering
 - Social network analysis
 - ...

Feature selection

- In generale non tutte le feature presenti in un campione sono rilevanti per il compito di clustering e, anzi, possono introdurre una certa «rumorosità» nei dati se prese in considerazione
- È necessario impiegare dei modelli di feature selection che cercano di individuare la «tendenza al clustering» di certi sottoinsiemi di feature

Feature selection

- Modelli basati su filtro: viene valutata la tendenza di una feature o di un gruppo di feature a contribuire al clustering utilizzando uno score numerico
- Modelli «wrapper»: approcci iterativi di clustering per tentativi su sottoinsiemi di feature
 - Computazionalmente onerosi, ma possono fornire una base per la scelta del miglior algoritmo di clustering per il problema sotto esame

Feature selection

- Modelli basati su filtro

$$\text{Term Strength} = P(t \in \bar{Y} | t \in \bar{X})$$

- È una *similarità* che può essere estesa a dati multidimensionali rappresentati come vettori binari di attributi presenti/assenti
- Analogamente si possono usare misure di rilevanza rispetto a dati quantitativi confrontando la distanza tra singoli attributi di due campioni rispetto alla distanza complessiva dei due campioni

$$\frac{d(\mathbf{x}_j^{(i)} - \mathbf{x}_j^{(k)})}{d(\mathbf{x}^{(i)} - \mathbf{x}^{(k)})}$$

Feature selection

- Modelli basati su filtro
 - Dipendenza predittiva da un attributo
 - Assunto: gli attributi rilevanti per il clustering sono correlati
 - Come fare a trovare gli attributi rilevanti?

Feature selection

- Modelli basati su filtro
 - Dipendenza predittiva da un attributo
 - *Idea*: eseguo una classificazione/regressione (a seconda del tipo di attributo) rispetto alla i -esima feature che funge da classe del campione
 - Spesso si usa la classificazione nearest neighbor
 - La misura di accuratezza della classificazione rispetto all'attributo i sarà la misura di rilevanza utilizzata per la feature selection

Feature selection

- Modelli basati su filtro
 - Entropia
 - I dati raggruppati in un cluster hanno entropia più bassa che quelli distribuiti uniformemente nello spazio di variazione
 - Discretizziamo i valori di ognuna delle k feature in ϕ intervalli
 - Otteniamo una griglia multidimensionale discreta composta da $m = \phi^k$ regioni

Feature selection

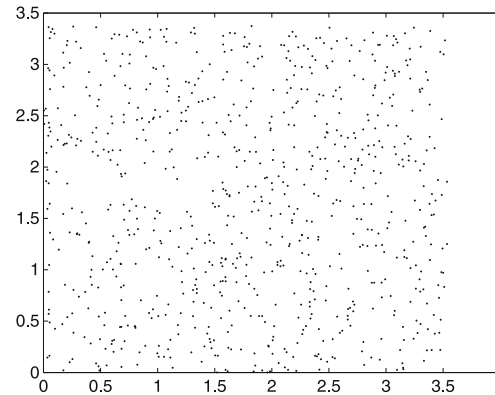
- Modelli basati su filtro
 - Entropia
 - Contiamo il numero di punti che cadono in ogni regione e, se p_i è la frazione di punti che cade nella regione i -esima, si ha:

$$E = - \sum_{i=1}^m [p_i \log(p_i) + (1 - p_i) \log(1 - p_i)]$$

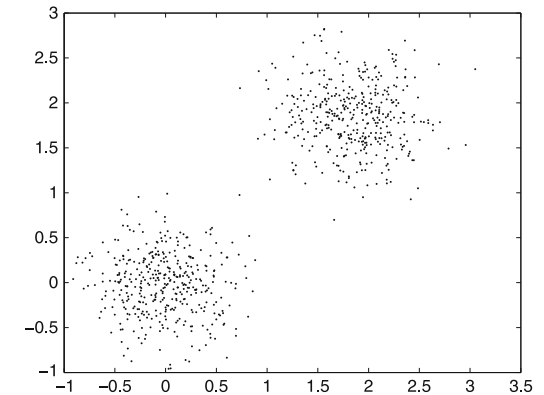
- Problemi di sparsità per elevate dimensioni

Feature selection

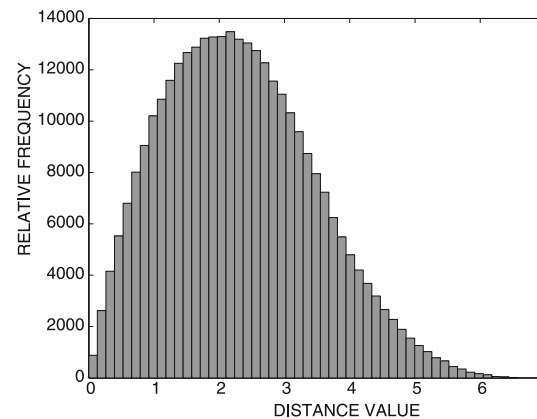
- Modelli basati su filtro
 - Entropia
 - *Le distanze 1-D tra le feature clusterizzano*
 - Entropia minore di quelle di punti non clusterizzati



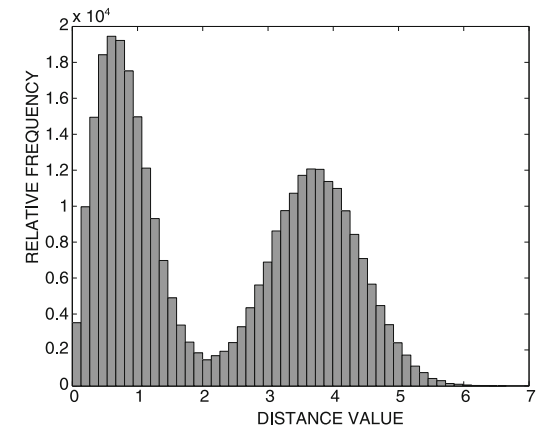
(a) Uniform Data



(b) Clustered data



(c) Distance distribution (uniform)



(d) Distance distribution (clustered)

Feature selection

- Modelli basati su filtro
 - Entropia
 - Si calcolano le «distanze punto a punto» tra i campioni lungo ogni feature e, se m sono gli intervalli di discretizzazione dei valori di distanza, si ha:

$$E = - \sum_{i=1}^m [p_i \log(p_i) + (1 - p_i) \log(1 - p_i)]$$

$$p_i = \frac{N_{d_i}}{N_d}, \quad d_i \in \text{i-th interval}, \quad i = 1, 2, \dots, m$$

Feature selection

- Modelli basati su filtro
 - Entropia
 - Si scelgono le feature con entropia minore
 - Approccio greedy: si eliminano le feature che portano via via alla maggiore diminuzione di E



Università
degli Studi
di Palermo



Feature selection

- Modelli basati su filtro
 - Hopkins Statistic
 - Si generi un campione $R = \{r_1, \dots, r_r\} \subset \mathcal{D}$
 - Si generi un campione sintetico $S = \{s_1, \dots, s_r\} \subset \mathbb{R}^d$, lo spazio dei dati
 - Siano $\alpha_i = \min_{\mathbf{x}} \text{Dist}(r_i, \mathbf{x})$ e $\beta_i = \min_{\mathbf{x}} \text{Dist}(s_i, \mathbf{x})$ $i = 1, \dots, r$ e $\mathbf{x} \in \mathcal{D} - R$

Feature selection

- Modelli basati su filtro

- Hopkins Statistic

$$H = \frac{\sum_{i=1}^r \beta_i}{\sum_{i=1}^r (\alpha_i + \beta_i)}$$

- $H \rightarrow 0.5$ per dati distribuiti uniformemente (α_i e β_i sono simili)
 - $H \rightarrow 1$ per dati clusterizzati ($\alpha_i \ll \beta_i$)

Feature selection

- Modelli wrapper
 - Si effettua una serie di clustering su differenti sottoinsiemi di feature e si verifica di volta in volta la qualità ottenuta con una apposita *misura interna di bontà del clustering*
 - Computazionalmente oneroso
 - Necessita di un approccio greedy per la ricerca dei sottoinsiemi rilevanti nello spazio delle feature
 - Fortemente dipendente dalla scelta della misura di bontà del clustering

Feature selection

- Modelli wrapper
 - Alternativamente si effettua un clustering sul sottoinsieme di feature considerate e si utilizzano «artificialmente» le etichette dei cluster ottenute come etichette di classe
 - Si utilizza un *criterio di bontà di classificazione supervisionata delle singole feature* su tali etichette per valutare indirettamente la bontà del clustering
 - Si selezionano le prime k feature con il valore migliore dell'indice di qualità di classificazione

Clustering basato su prototipi rappresentativi

Algorithm *GenericRepresentative*(Database: $\mathcal{D}$, Number of Representatives: k)

begin

Initialize representative set S ;

repeat

Create clusters $(\mathcal{C}_1 \dots \mathcal{C}_k)$ by assigning each point in $\mathcal{D}$ to closest representative in S using the distance function $Dist(\cdot, \cdot)$;

Recreate set S by determining one representative $\bar{Y}_j$ for each $\mathcal{C}_j$ that minimizes $\sum_{\bar{X}_i \in \mathcal{C}_j} Dist(\bar{X}_i, \bar{Y}_j)$;

until convergence;

return $(\mathcal{C}_1 \dots \mathcal{C}_k)$;

end

Passo di assegnazione

Passo di ottimizzazione

Clustering basato su prototipi rappresentativi

- I k cluster sono in genere un iperparametro determinato dall'utente
- Si devono determinare k prototipi rappresentativi $\bar{Y}_j$ che minimizzino la funzione obiettivo

$$O = \sum_{j=1}^k \sum_{\bar{X}_i \in \mathcal{C}_j} Dist(\bar{X}_i, \bar{Y}_j)$$

- I valori degli $\bar{Y}_j$ in generale *non appartengono* al data set $\mathcal{D}$
- La scelta di diverse forme di $Dist(.,.)$ dà luogo a differenti algoritmi

Clustering basato su prototipi rappresentativi

- K-means

$$Dist(\overline{X_i}, \overline{Y_j}) = \|\overline{X_i} - \overline{Y_j}\|_2^2$$

- Si può mostrare che i valori ottimi degli $\bar{Y}_j$ sono i valori medi dei punti in ogni singolo cluster $\mathcal{C}_j$

Clustering basato su prototipi rappresentativi

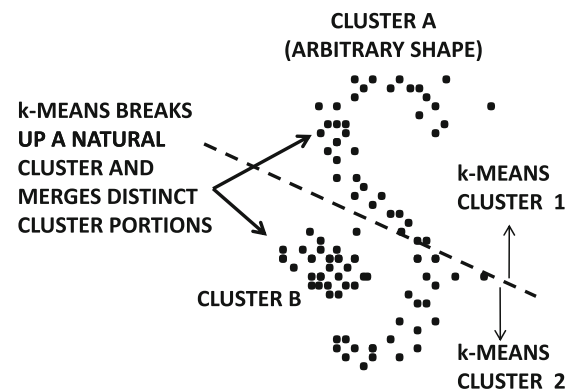
- Mahalanobis K-means

$$Dist(\overline{X}_i, \overline{Y}_j) = (\overline{X}_i - \overline{Y}_j) \Sigma_j^{-1} (\overline{X}_i - \overline{Y}_j)^T$$

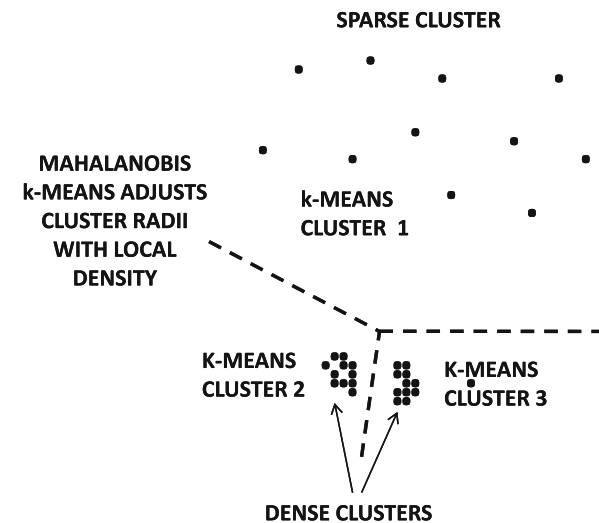
- La distanza di Mahalanobis è opportuna per cluster di forma (iper-)ellittica
- La covarianza locale Σ_j , calcolata sui punti del cluster $\mathcal{C}_j$:
 - fornisce una misura implicita di densità del cluster e quindi consente di gestire cluster a densità variabile
 - Consente la gestione di cluster con forme (iper-)ellittiche variabili

Clustering basato su prototipi rappresentativi

- *E quando la forma non è più una sfera o un'ellissoide?*



(a) Varying cluster shape
(Bad for k -means)



(b) Varying cluster density
(Good for Mahalanobis k -means)

Clustering basato su prototipi rappresentativi

- Per la gestione di cluster di forma qualunque, il k-means ricorre al cosiddetto *kernel trick*
 - Si consideri una funzione $\Phi(.)$ che mappa un vettore in uno spazio a più alta dimensionalità
 - Si definisce «kernel» una qualunque funzione tale che:

$$K(\overline{X_i}, \overline{X_j}) = \Phi(\overline{X_i}) \cdot \Phi(\overline{X_j})$$

Clustering basato su prototipi rappresentativi

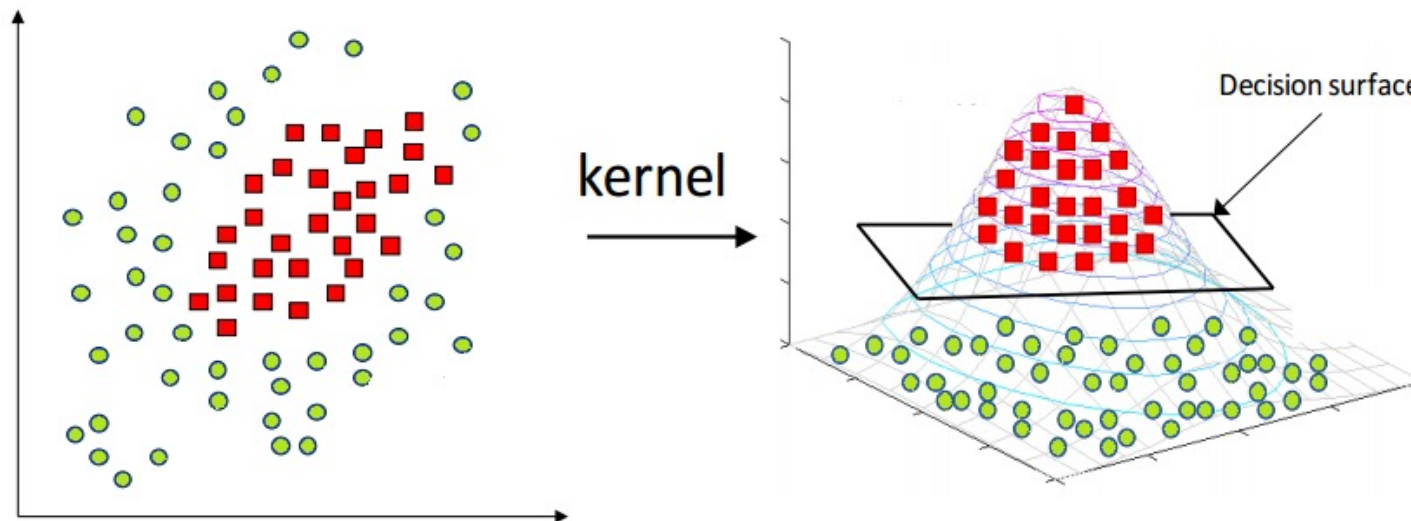
- Per la gestione di cluster di forma qualunque, il k-means ricorre al cosiddetto *kernel trick*
- L'uso del kernel è un operatore lineare nello spazio trasformato e quindi consente l'uso di algoritmi lineari nello spazio definito da $\Phi(\cdot)$
- Il kernel $K(\cdot, \cdot)$ viene definito direttamente in termini dei vettori dello spazio di ingresso e non è *mai* necessario calcolare il mapping $\Phi(\cdot)$

$$K(\overline{X_i}, \overline{X_j}) = \Phi(\overline{X_i}) \cdot \Phi(\overline{X_j})$$

→ Uso di *euristiche* e di *algoritmi di ML dedicati* per selezionare il kernel K più adatto ad un dato compito

Clustering basato su prototipi rappresentativi

- Per la gestione di cluster di forma qualunque, il k-means ricorre al cosiddetto *kernel trick*



Clustering basato su prototipi rappresentativi

- K-medians

$$Dist(\overline{X_i}, \overline{Y_j}) = \|\overline{X_i} - \overline{Y_j}\|_1$$

- Uso della distanza di Manhattan
- Robusto rispetto agli outlier a causa dell'uso dei mediani

Clustering basato su prototipi rappresentativi

- K-medoids
- Impone che i valori degli $\bar{Y}_j$ appartengano a $\mathcal{D}$
- Adattabile a dati eterogenei con la giusta definizione dell'appropriata misura di distanza/similarità
- Computazionalmente oneroso

Algorithm *GenericMedoids*(Database: $\mathcal{D}$, Number of Representatives: k)
begin

Initialize representative set S by selecting from $\mathcal{D}$;

repeat

Create clusters $(\mathcal{C}_1 \dots \mathcal{C}_k)$ by assigning
each point in $\mathcal{D}$ to closest representative in S
using the distance function $Dist(\cdot, \cdot)$;

Determine a pair $\bar{X}_i \in \mathcal{D}$ and $\bar{Y}_j \in S$ such that
replacing $\bar{Y}_j \in S$ with $\bar{X}_i$ leads to the
greatest possible improvement in objective function;

Perform the exchange between $\bar{X}_i$ and $\bar{Y}_j$ only
if improvement is positive;

until no improvement in current iteration;

return $(\mathcal{C}_1 \dots \mathcal{C}_k)$;

end

Clustering gerarchico

- Il clustering gerarchico crea una tassonomia di cluster che vengono aggregati attraverso il calcolo di distanze tra essi
- Il calcolo di tali distanze utilizza in genere altri approcci come quelli basati sul concetto di densità o sull'utilizzo di grafi
- Approcci
 - Agglomerativi o bottom-up
 - Divisivi o top-down

Clustering gerarchico

- Approcci agglomerativi

Algorithm *AgglomerativeMerge*(Data: $\mathcal{D}$)

begin

Initialize $n \times n$ distance matrix M using $\mathcal{D}$;

repeat

Pick closest pair of clusters i and j using M ;

Merge clusters i and j ;

Delete rows/columns i and j from M and create
a new row and column for newly merged cluster;

Update the entries of new row and column of M ;

until termination criterion;

return current merged cluster set;

end

M è la struttura dati su cui lavorano tutti gli algoritmi gerarchici

Due approcci:

1. Soglia massima sulla distanza tra due cluster da fondere
2. Soglia minima sul numero di cluster

Come calcolo la distanza tra due cluster i e j che contengono rispettivamente m_i e m_j elementi??

Clustering gerarchico

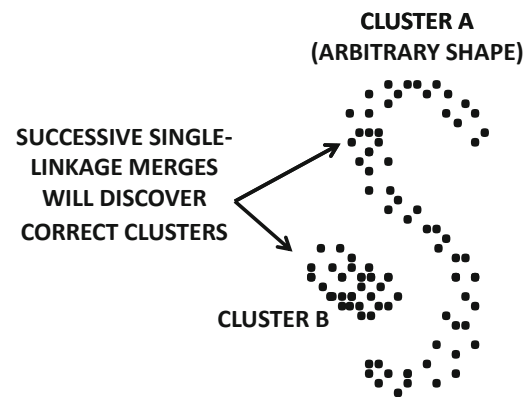
- Stime di distanza per approcci agglomerativi, dopo aver fuso i cluster i e j
 - Single linkage – la distanza è la minima tra le $m_i \times m_j$ distanze possibili
 - Per ogni cluster $k \neq i, j$ si considerano $\min\{M_{ik}, M_{jk}\}$ per le righe e $\min\{M_{ki}, M_{kj}\}$ per le colonne di M
 - Complete linkage – la distanza è la massima tra le $m_i \times m_j$ distanze possibili
 - Per ogni cluster $k \neq i, j$ si considerano $\max\{M_{ik}, M_{jk}\}$ per le righe e $\max\{M_{ki}, M_{kj}\}$ per le colonne di M

Clustering gerarchico

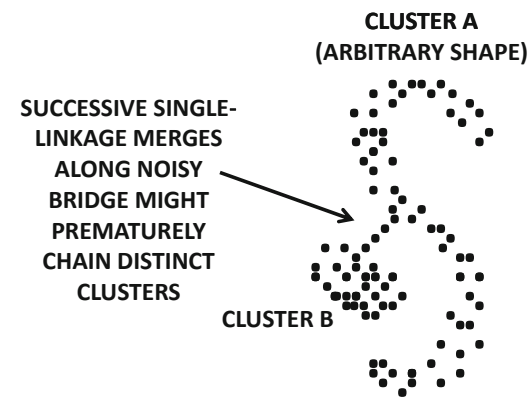
- Stime di distanza per approcci agglomerativi, dopo aver fuso i cluster i e j
 - Group-average linkage – la distanza è la media tra le $m_i \times m_j$ distanze possibili
 - Per ogni cluster $k \neq i, j$ si considerano $(m_i M_{ik} + m_j M_{jk}) / (m_i + m_j)$ per le righe e $(m_i M_{ki} + m_j M_{kj}) / (m_i + m_j)$ per le colonne di M
 - Centroidi più vicini – si fondono i cluster con i centroidi più vicini

Clustering gerarchico

- Approcci agglomerativi: limiti del clustering



(a) Good case with no noise



(b) Bad case with noise

Clustering gerarchico

- Approcci divisivi
 - $\mathcal{A}$ può essere un algoritmo di clustering qualunque
 - Bisecting k-means: esegue un 2-means ad ogni iterazione
 - La scelta del nodo L può essere dettata da diversi criteri
 - Il nodo più vicino alla radice

Algorithm *GenericTopDownClustering*(Data: $\mathcal{D}$, Flat Algorithm: $\mathcal{A}$)
begin
 Initialize tree $\mathcal{T}$ to root containing $\mathcal{D}$;
 repeat
 Select a leaf node L in $\mathcal{T}$ based on pre-defined criterion;
 Use algorithm $\mathcal{A}$ to split L into $L_1 \dots L_k$;
 Add $L_1 \dots L_k$ as children of L in $\mathcal{T}$;
 until termination criterion;
end

Clustering probabilistico

- *Soft Clustering*: ogni punto ha una probabilità anche piccola, ma diversa da 0, di appartenere ad *ogni* cluster
- Considereremo un *modello generativo* $\mathcal{M}$ che descrive la probabilità p_{data} di generare un punto appartenente ad un dato cluster
- Si assumerà che p_{data} sia una mistura di distribuzioni $\mathcal{G}_i$, ovvero i nostri cluster, con un insieme di prior $\alpha_i = P(\mathcal{G}_i)$

Clustering probabilistico

- La massimizzazione della likelihood si ottiene con un approccio iterativo a due passi detto *Expectation Maximization (EM)* che ci consente di stimare l'insieme $\mathbf{w}$ di tutti i parametri della mistura ed i prior α_i
- Usiamo lo schema del Teorema di Bayes

$$P(\mathbf{w}|\mathcal{D}) = \frac{P(\mathcal{D}|\mathbf{w})P(\mathbf{w})}{P(\mathcal{D})}$$

Clustering probabilistico

- Expectation Maximization

$$P(\mathbf{w}|\mathcal{D}) = \frac{P(\mathcal{D}|\mathbf{w})P(\mathbf{w})}{P(\mathcal{D})}$$

- Expectation: si effettua una stima di $P(\mathbf{w}|\mathcal{D})$ delle predizioni del modello fissati i parametri $\mathbf{w}$
- Maximization: si effettua uno step di massimizzazione di $P(\mathcal{D}|\mathbf{w})$ per calcolare il nuovo vettore dei parametri ponendo a 0 la derivata rispetto a $\mathbf{w}$

Clustering probabilistico

- Applichiamo l'algoritmo EM al nostro modello
- Expectation - stima di $P(\mathcal{G}_i | X_j, \mathbf{w})$ in ogni punto tramite il Teorema di Bayes
- Maximization – nuova stima dei parametri α_i e $\mathbf{w}$
 - α_i viene stimato come valor medio delle $P(\mathcal{G}_i | X_j, \mathbf{w})$ per tutti i punti del cluster i
 - Si aggiorna $\mathbf{w}$ massimizzando la log-likelihood $\mathcal{L}$: $\partial \mathcal{L}(\mathcal{D} | \mathbf{w}) / \partial \mathbf{w} = 0$

Clustering probabilistico

- Si consideri una mistura di gaussiane in d dimensioni:

$$f^{i,\mathbf{w}}(\overline{X_j}) = \frac{1}{\sqrt{|\Sigma_i|}(2 \cdot \pi)^{(d/2)}} e^{-\frac{1}{2}(\overline{X_j} - \overline{\mu_i})\Sigma_i^{-1}(\overline{X_j} - \overline{\mu_i})}$$

- Si può mostrare che il passo di expectation in questo caso pesa i cluster in modo da assegnare i punti ai cluster *esattamente* come il Mahalanobis K-means
- Nel caso particolare in cui $\alpha_i = 1/k$ e tutte le componenti hanno deviazione standard uguale a σ si ottiene il k-means classico

Clustering per densità

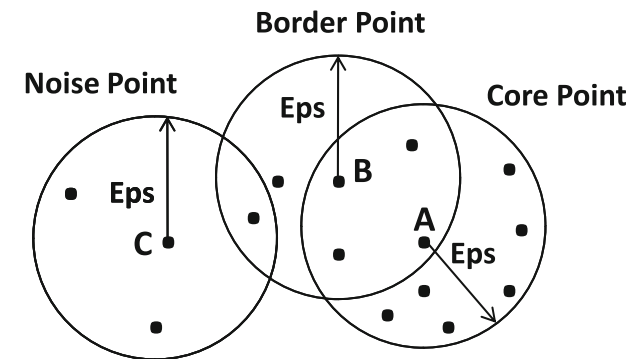
- In questi approcci si cerca di creare i cluster come componenti connesse di un grafo costituito da elementi strutturali che sono caratterizzati dal possedere un parametro di *densità* sopra una certa soglia
- Approcci *density based* propriamente detti: gli elementi strutturali sono punti il cui intorno contiene un numero di punti maggiore di una certa soglia

Clustering per densità

- In questi approcci si cerca di creare i cluster come componenti connesse di un grafo costituito da elementi strutturali che sono caratterizzati dal possedere un parametro di *densità* sopra una certa soglia
- Approcci *grid-based*: gli elementi strutturali sono ipercubi di una griglia che suddivide lo spazio dei dati: un ipercubo sarà «denso» se contiene un numero di punti maggiore di una certa soglia

Clustering per densità

- Algoritmi density based – DBSCAN
 - DBSCAN si basa sulla definizione di tre tipologie di punti nel data set
 - Core points: punti che nel loro intorno di dato raggio *Eps* contengono almeno τ punti
 - Border points: punti che contengono meno di τ punti nel loro intorno, ma contengono un core point
 - Noise points: punti che nel loro intorno contengono meno di τ punti e non contengono core point



Clustering per densità

- Algoritmi density based – DBSCAN

Algorithm *DBSCAN*(Data: $\mathcal{D}$, Radius: Eps , Density: τ)

begin

Determine core, border and noise points of $\mathcal{D}$ at level (Eps, τ) ;

Create graph in which core points are connected
if they are within Eps of one another;

Determine connected components in graph;

Assign each border point to connected component
with which it is best connected;

return points in each connected component as a cluster;

end

*Si devono calcolare le
distanze tra tutte le
coppie di punti!!*

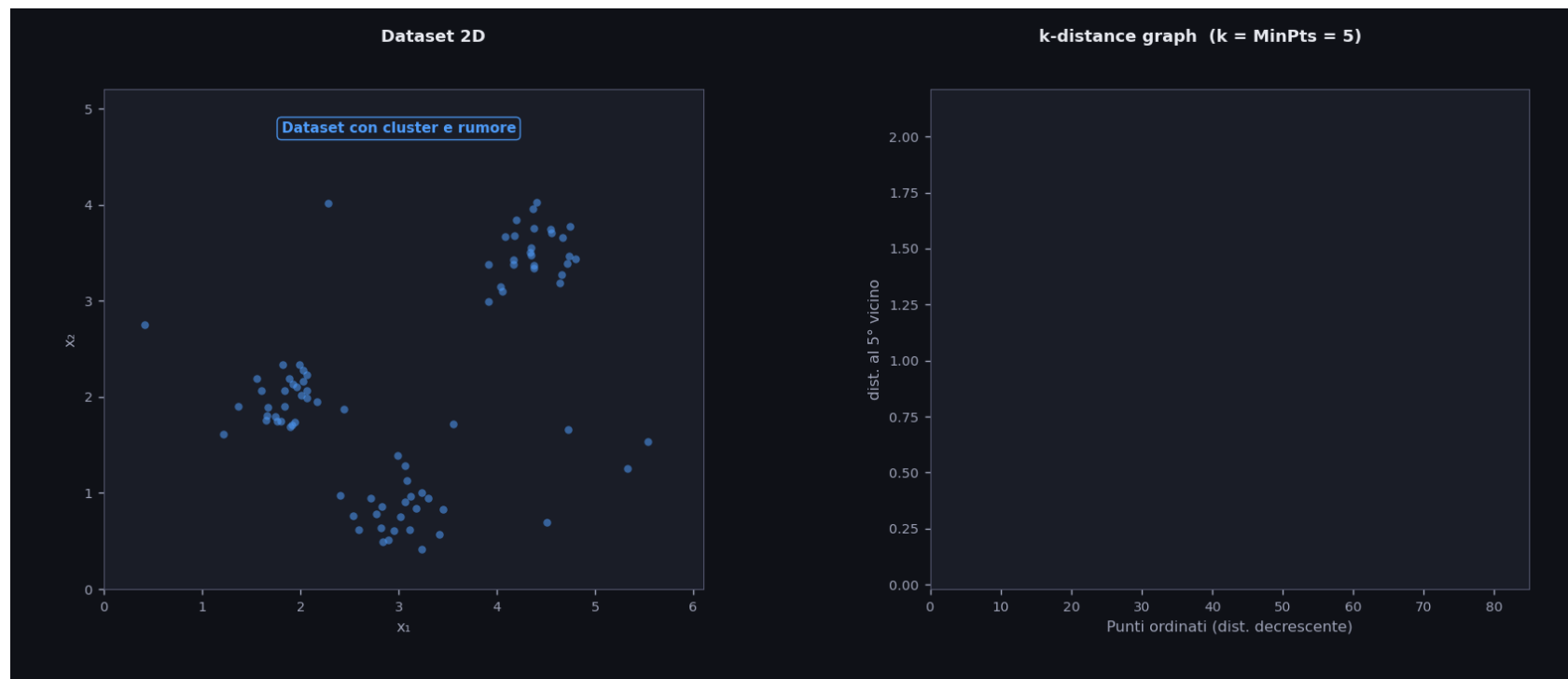
Clustering per densità

- Algoritmi density based – DBSCAN
 - Il passo di determinazione del grafo dei core points è un approccio single linkage con distanza Eps
 - La ricerca dei core points è un problema $O(n^2)$ che può essere ridotto a $O(n \log n)$ con l'utilizzo di apposite strutture di indicizzazione
 - *Problemi legati all'elevata dimensionalità poiché tali strutture richiedono calcoli di distanze tra i punti che sono onerose e poco significative per elevati valori di d*

Clustering per densità

- Algoritmi density based – DBSCAN
 - Il fatto che τ (denominata in genere *MinPts* nella letteratura di settore) sia fissato comporta problemi nella gestione di cluster a densità variabile
 - *Eps* e τ sono in relazione tra loro: *Eps* può essere determinato come valore di cutoff della distanza di ogni punto dai suoi τ nearest neighbors

Clustering per densità



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Clustering per densità

- Algoritmi grid based

Algorithm *GenericGrid*(Data: $\mathcal{D}$, Ranges: p , Density: τ)

begin

Discretize each dimension of data $\mathcal{D}$ into p ranges;

Determine dense grid cells at density level τ ;

Create graph in which dense grids are connected if they are adjacent;

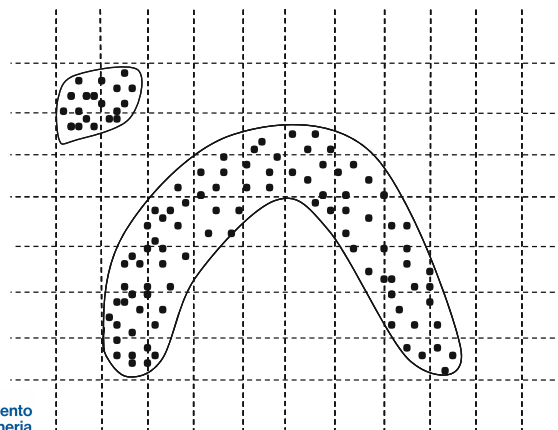
Determine connected components of graph;

return points in each connected component as a cluster;

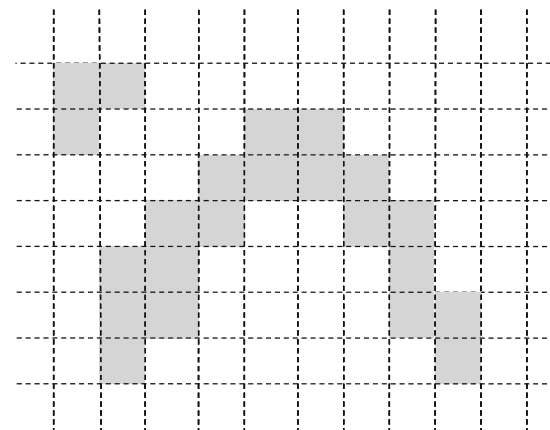
end

Otterremo p^d celle!!

Le distanze non servono più: si calcolano i confronti con le posizioni discretizzate delle celle!!



(a) Data points and grid



(b) Agglomerating adjacent grids

Clustering per densità

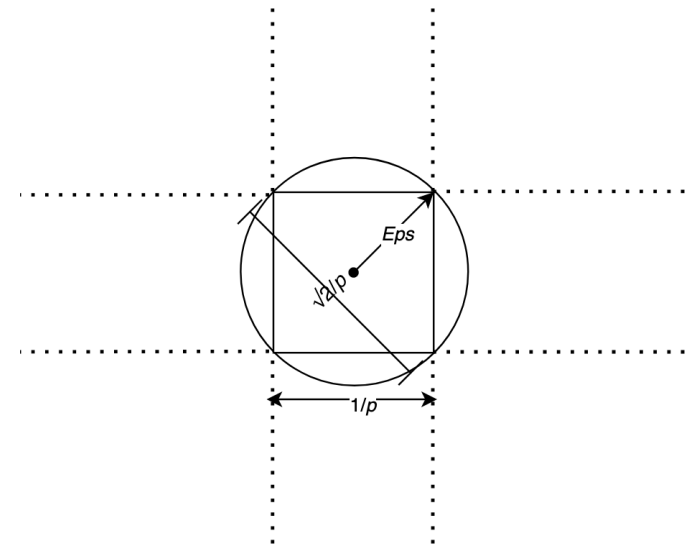
- Algoritmi grid based
 - Il clustering dipende dal numero delle celle p lungo le dimensioni della griglia e dal parametro di densità $MinPts$
 - Celle troppo grandi (p piccolo) possono contenere punti di diversi cluster, mentre celle troppo piccole danno luogo a un numero elevato di celle vuote
 - Un valore di $MinPts$ troppo basso fa collassare più celle in un cluster, mentre al contrario tende a spezzare i cluster
 - *Problemi ad elevata dimensionalità perché il numero delle celle cresce esponenzialmente con d*

Clustering per densità

- Algoritmi grid based
 - È possibile legare la dimensione della cella a al valore di Eps , a parità di scelta di $MinPts$, per ottenere forme dei cluster simili a quelle di DBSCAN
 - Assumiamo che i dati si trovino all'interno di un ipercubo unitario:

$$\frac{1}{p} \propto \frac{Eps}{\sqrt{2}}$$

Otteniamo un'adiacenza tra celle dense che raggruppa i punti in modo molto simile agli intorno dei core point



Misure di bontà del clustering

- Il clustering è una procedura non supervisionata e quindi non ci fornisce, in principio, informazioni utili alla validazione della bontà del task effettuato
- Si determinano comunque due tipologie di criteri
 - Criteri interni, basati sui dati
 - Criteri esterni, quando delle etichette di classe sono comunque disponibili per verificare se il clustering ha seguito il criterio di classificazione

Misure di bontà del clustering

- Criteri di validazione interni
 - Within Cluster Sum of Squares (WCSS)

$$WCSS = \sum_{j=1}^K \sum_{\bar{X}_i \in \mathcal{C}_j} (\bar{X}_i - \bar{Y}_j)^2$$

- Pensata principalmente per il k-means e per gli algoritmi basati su distanze e cluster di forma sferica
- Tende comunque a diminuire all'aumentare del numero di cluster

Misure di bontà del clustering

- Criteri di validazione interni

- Rapporto delle distanze Intra-cluster/inter-cluster

$$Intra = \sum_{(\bar{X}_i, \bar{X}_j) \in P} \text{dist}(\bar{X}_i, \bar{X}_j) / |P|$$

- Campioniamo r punti da $\mathcal{D}$ di cui P appartengono ad un certo cluster e i rimanenti Q no

$$Inter = \sum_{(\bar{X}_i, \bar{X}_j) \in Q} \text{dist}(\bar{X}_i, \bar{X}_j) / |Q|$$

- Il clustering è migliore quanto più $Intra/Inter$ è basso

Misure di bontà del clustering

- Criteri di validazione interni
 - Silhouette coefficient

$Davg_i^{in} = \text{avg} \left(\text{Dist} \left(\overline{X}_k, \overline{X}_l \right) \right), \overline{X}_k, \overline{X}_l \in \mathcal{C}_i$ Distanza media tra i punti del cluster i

$Dmin_i^{out} = \min_{\mathcal{C}_m} \text{avg} \left(\text{Dist} \left(\overline{X}_k, \overline{X}_h \right) \right), \overline{X}_k \in \mathcal{C}_i, \overline{X}_h \in \mathcal{C}_m$ Minimo tra le distanze medie degli altri cluster dal cluster i

Misure di bontà del clustering

- Criteri di validazione interni
 - Silhouette coefficient

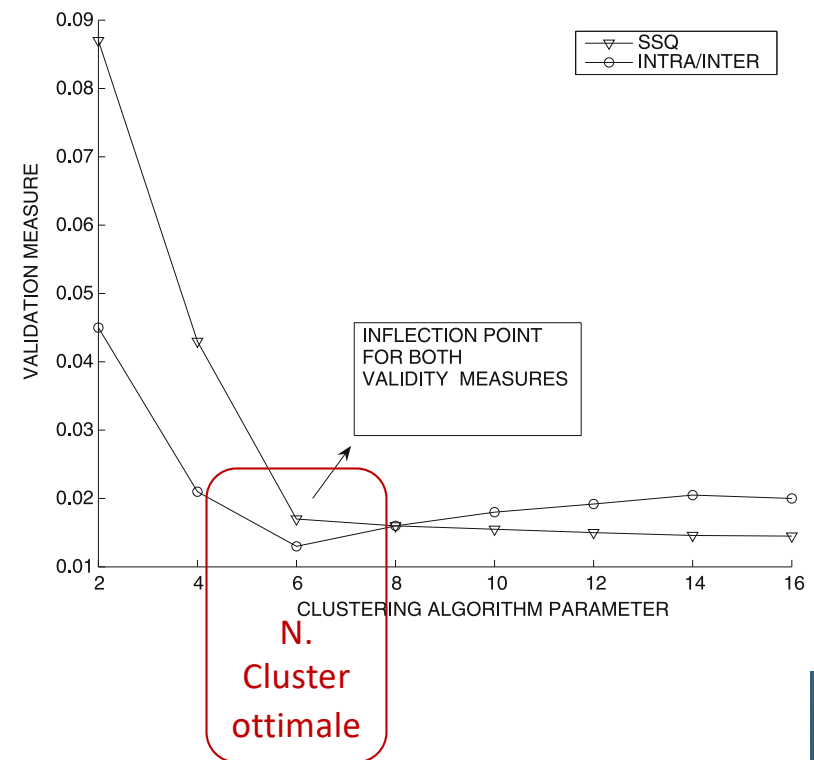
$$S_i = \frac{Dmin_i^{out} - Davg_i^{in}}{\max \{ Dmin_i^{out}, Davg_i^{in} \}}$$

$$S = \frac{1}{n} \sum_i S_i$$

- S varia in $[-1, 1]$: valori grandi positivi indicano buona separazione tra i cluster, mentre valori negativi indicano mescolamento

Misure di bontà del clustering

- Criteri di validazione interni
 - Possono essere utilizzati anche per effettuare il tuning dei parametri e cioè stabilire il numero di cluster
 - Elbow method: sia *WCSS* sia *Intra/Inter* decrescono rapidamente fino al numero ottimale di cluster e poi si crea un plateau



Misure di bontà del clustering

- Criteri di validazione interni
 - Tutti questi criteri sottendono ciascuno un modello particolare di cluster che potrebbe non essere quello reale
 - Questo rende difficile anche il paragone tra i diversi criteri usati per testare la bontà di un dato algoritmo

Misure di bontà del clustering

- Criteri di validazione esterni
 - Si utilizzano le etichette di cluster, ove disponibili, da usare come etichette di classe
 - Disponibilità di benchmark data set
 - Oppure dati reali con etichette di classe che dipendono dall'applicazione di classificazione e potrebbero non riflettere le caratteristiche dei cluster naturali dei dati

Misure di bontà del clustering

- Criteri di validazione esterni
 - Siano k_t le etichette di classe e k_d il numero di cluster determinati dall'algoritmo
 - Se $k_t = k_d$ si può utilizzare la *matrice di confusione*

		Valori predetti			
Valori reali	Cluster Indices	1	2	3	4
	1	97	0	2	1
	2	5	191	1	3
	3	4	3	87	6
	4	0	0	5	195

		1	2	3	4
1	Cluster Indices	33	30	17	20
2		51	101	24	24
3		24	23	31	22
4		46	40	44	70

Misure di bontà del clustering

- Criteri di validazione esterni
 - In generale si usano degli indici numerici
 - Sia m_{ij} il numero di punti della classe i mappati nel cluster j , cioè l'elemento (i,j) della matrice di confusione

$$N_i = \sum_{j=1}^{k_d} m_{ij}, \quad \forall i = 1 \dots k_t$$

Somma dei valori lungo una riga:
l'effettivo numero di punti
appartenenti alla classe i

$$M_j = \sum_{i=1}^{k_t} m_{ij}, \quad \forall j = 1 \dots k_d$$

Somma dei valori lungo una colonna:
il numero di punti predetti
come appartenenti al cluster j

Misure di bontà del clustering

- Criteri di validazione esterni
 - Purity: si assume che un buon cluster avrà quanti più possibili punti appartenenti tutti ad una stessa classe

$$\text{Purity} = \frac{\sum_{j=1}^{k_d} P_j}{\sum_{j=1}^{k_d} M_j}, \quad P_j = \max_i m_{ij}$$

Numero di punti predetti nella classe dominante

- La Purity tende a 1 per un buon clustering

Misure di bontà del clustering

- Criteri di validazione esterni

- Gini index: considera il contributo anche delle altre classi presenti nel cluster
- È un buon cluster quello in cui i punti sono concentrati in poche classi e c'è poca dispersione
- G_j tende a 0 per un buon clustering, mentre ha $1 - 1/k_t$ come limite superiore

$$G_j = 1 - \sum_{i=1}^{k_t} \left(\frac{m_{ij}}{M_j} \right)^2$$

% punti di
classe i
predetti
nel cluster j

$$G_{average} = \frac{\sum_{j=1}^{k_d} G_j \cdot M_j}{\sum_{j=1}^{k_d} M_j}$$

Misure di bontà del clustering

- Criteri di validazione esterni
 - Entropia: l'indice di Gini G_j è legato all'entropia E_j del cluster j poiché m_{ij}/M_j è la probabilità frequentista che un punto di classe i appartenga al cluster j

$$E_j = - \sum_{i=1}^{k_t} \left(\frac{m_{ij}}{M_j} \right) \cdot \log \left(\frac{m_{ij}}{M_j} \right)$$

$$E_{\text{average}} = \frac{\sum_{j=1}^{k_d} E_j \cdot M_j}{\sum_{j=1}^{k_d} M_j}$$

Misure di bontà del clustering

- Criteri di validazione esterni
 - È possibile infine stabilire una misura di bontà del clustering analizzando la matrice di confusione con le misure tipiche della classificazione: Precision, Recall e F_1 -score (che vedremo più avanti)

Classificazione

- Dato un insieme di dati di addestramento, ciascuno associato ad una «etichetta» che individua l'appartenenza del dato ad una classe, *la classificazione consiste nel predire il valore dell'etichetta per dati di test mai visti dall'algoritmo*
- È una classe di algoritmi di apprendimento supervisionati
 - Le etichette provengono da una associazione artificiale (dipendente dall'applicazione) ai dati e non dalla naturale tendenza di questi ultimi a formare cluster
 - È, forse, la tipologia di algoritmo di ML più comune

Selezione delle feature

- Filtri: indicatori numerici della rilevanza delle feature
- Modelli «wrapped»:
 - un algoritmo di classificazione viene usato per valutare la performance su un sotto-insieme di feature e quindi «avvolge» il vero e proprio algoritmo di classificazione in uno schema di ricerca delle feature rilevanti
- Modelli «embedded»:
 - l'algoritmo stesso fornisce indicazioni sulle feature rilevanti e, dopo averle individuate, viene riaddestrato solo su di esse

Selezione delle feature

- Filtri

- Usati per dati categorici

$$G(v_i) = 1 - \sum_{j=1}^k p_j^2$$

Valore i-esimo dell'attributo categorico a r valori

Frazione dei punti che hanno il valore v_i nella classe j

- Gini index

$G(v_i) = 0$ se tutti i valori v_i appartengono alla stessa classe

$$G = \sum_{i=1}^r n_i G(v_i) / n$$

Frazione dei punti che hanno il valore v_i

Selezione delle feature

- Filtri
 - Usati per dati categorici

- Entropia

$$E(v_i) = - \sum_{j=1}^k p_j \log_2(p_j), \quad E = \sum_{i=1}^r n_i E(v_i) / n$$

Frazione dei punti
che hanno il valore
 v_i nella classe j

Selezione delle feature

- Filtri

- Fisher score

- Usato per attributi numerici

- Per ogni feature:

- $p_j \rightarrow$ frazione dei campioni appartenete alla classe j
- $\mu_j \rightarrow$ media dell'attributo nella classe j
- $\mu \rightarrow$ media totale dell'attributo nel data set
- $\sigma_j \rightarrow$ dev. standard dell'attributo nella classe j

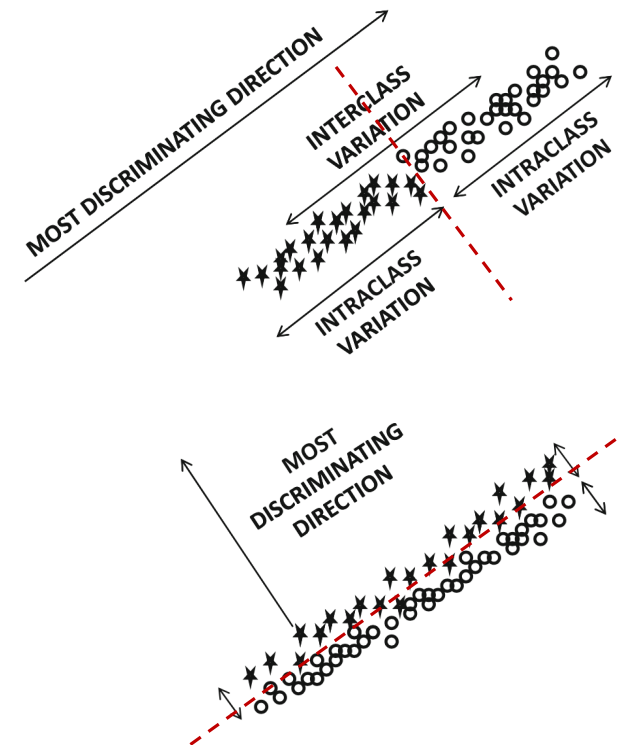
$$F = \frac{\sum_{j=1}^k p_j (\mu_j - \mu)^2}{\sum_{j=1}^k p_j \sigma_j^2}$$

Separazione media «inter-classe»

Separazione media «intra-classe»

Selezione delle feature

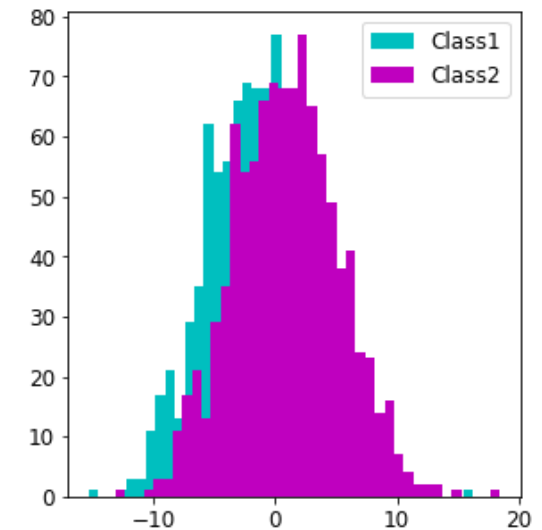
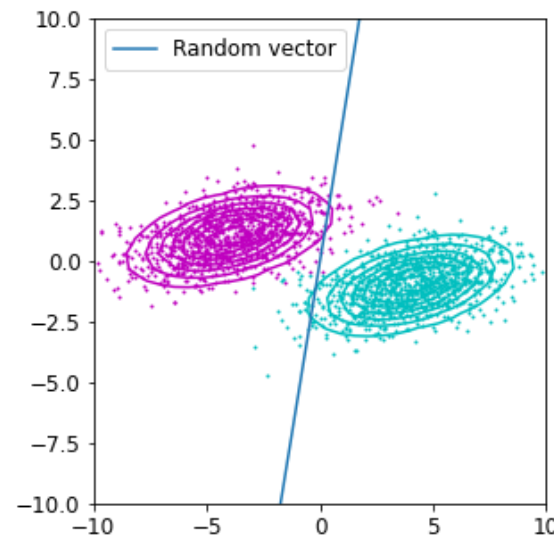
- Filtri
 - Fisher Linear Discriminant
 - Generalizzazione del Fisher Score per combinazioni lineari di feature numeriche
 - Tende a trovare, in forma supervisionata, la direzione di massima variazione delle feature e, per contro, l'iperpiano perpendicolare che separa meglio le classi rispetto alle feature
 - Il rapporto *Inter-class/Intra-class* tende ad essere massimizzato



Selezione delle feature

- Filtri

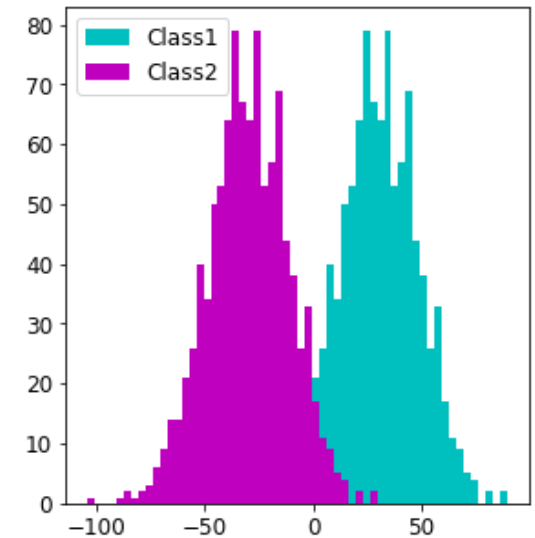
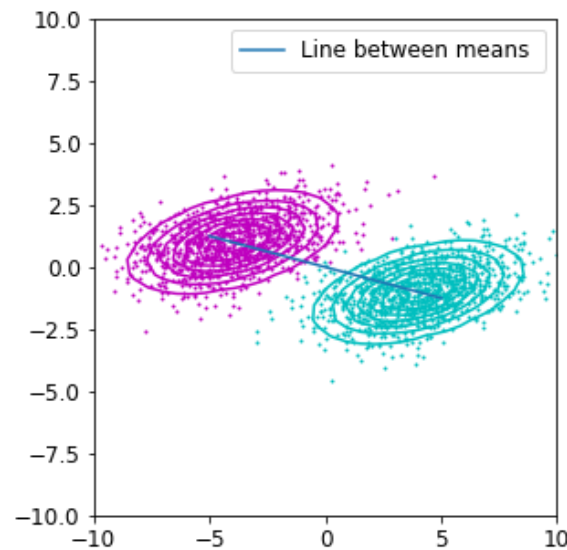
- Fisher Linear Discriminant: caso binario
- Troviamo la direzione lungo la quale le *proiezioni* dei punti del data set risultano *massimamente separati* rispetto alle due classi



Selezione delle feature

- Filtri

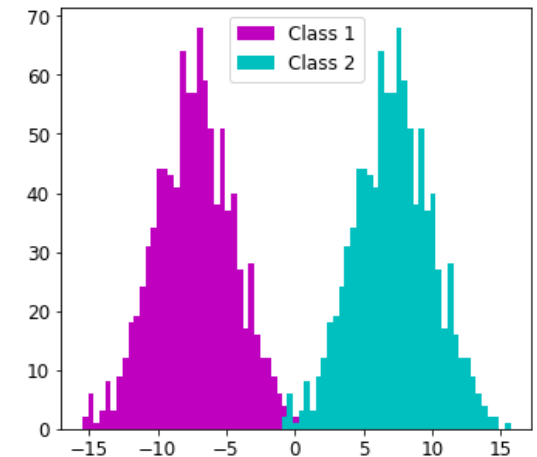
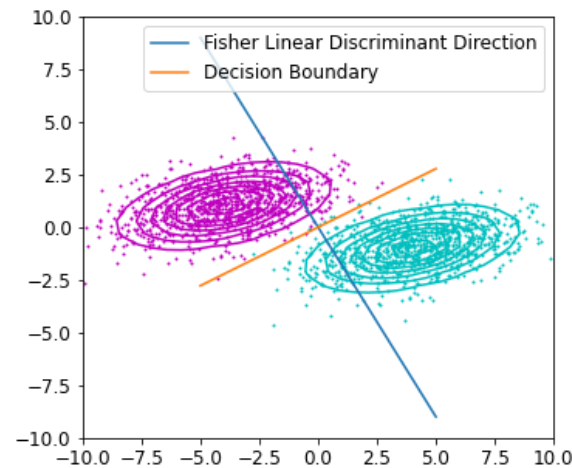
- Fisher Linear Discriminant: caso binario
- Troviamo la direzione w lungo la quale massimizziamo le *proiezioni dei punti medi* di ciascuna classe
- Bene, ma non benissimo: perché? → *Dobbiamo tenere conto della varianza dei dati proiettati lungo w !!*



Selezione delle feature

- Filtri

- Fisher Linear Discriminant: caso binario
- Tenendo conto delle varianze intra-classe s_1 e s_2 troviamo la direzione lungo la quale massimizziamo la *separazione dei punti* di ciascuna classe
- Massimizziamo una funzione obiettivo analoga al rapporto *Inter/Intra* lungo la direzione w



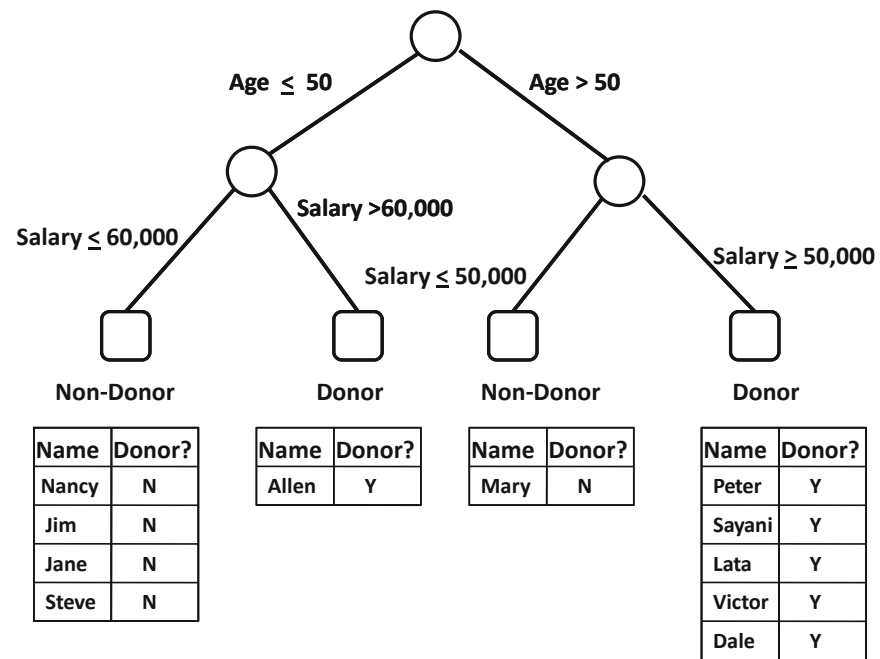
Selezione delle feature

- Modelli «wrapped»
 - Si parte da un insieme di feature $F = \{\}$
 - Si aggiungono feature a F e si testa l'accuratezza di un algoritmo di classificazione $\mathcal{A}$ per accettare l'aggiunta delle nuova feature a F
 - L'incremento di F si può fare secondo diverse strategie
 - Random
 - Aggiunta della feature con maggior potere discriminativo rispetto ad un criterio di filtro

Decision Tree

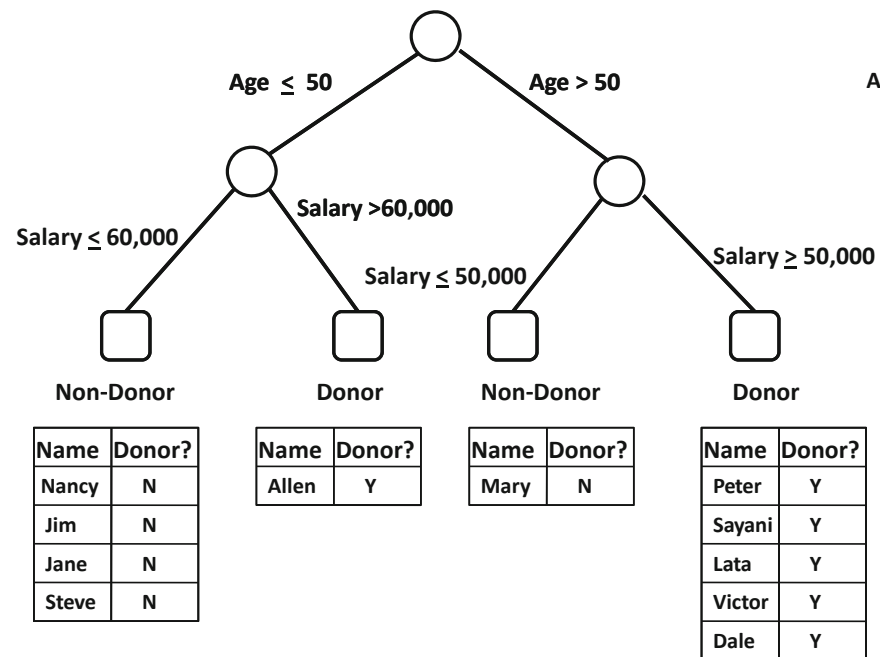
- Il data set viene suddiviso ricorsivamente in parti più piccole sulla base del discriminare sui valori degli attributi
- I nodi foglia dell'albero vengono attribuiti alla classe dominante
 - Gerarchico similmente al clustering
 - Split univariato → decisione su un solo attributo
 - Split multivariato → decisione su un insieme di attributi

Decision tree

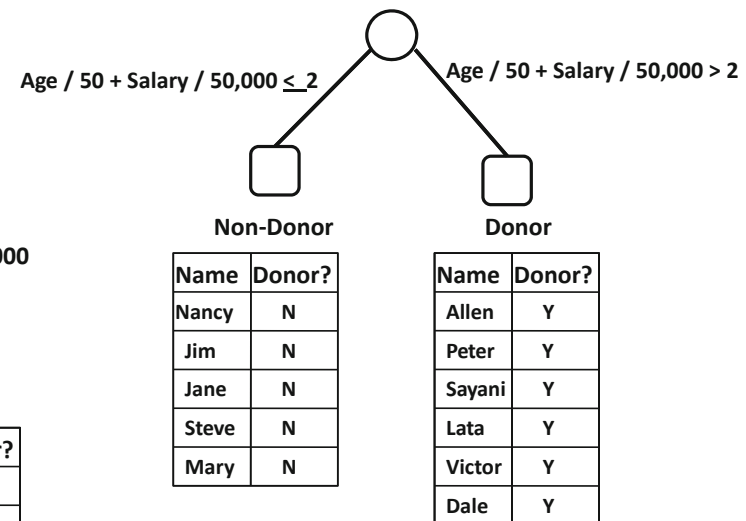


(a) Univariate split

Decision tree



(a) Univariate split



(b) Multivariate split

Decision tree

Algorithm *GenericDecisionTree*(Data Set: $\mathcal{D}$)

begin

Create root node containing $\mathcal{D}$;

repeat

Select an eligible node in the tree;

Split the selected node into two or more nodes
based on a pre-defined split criterion;

until no more eligible nodes for split;

Prune overfitting nodes from tree;

Label each leaf node with its dominant class;

end

- Split binario
- Split a r vie per attributi categorici o per attributi numerici con piccolo range di variazione
- Split binario per attributi categorici binarizzati
- Split a soglia per attributi numerici

Decision tree

Algorithm *GenericDecisionTree*(Data Set: $\mathcal{D}$)

begin

Create root node containing $\mathcal{D}$;

repeat

Select an eligible node in the tree;

Split the selected node into two or more nodes
based on a pre-defined **split criterion**;

until no more eligible nodes for split;

Prune overfitting nodes from tree;

Label each leaf node with its dominant class;

end

- Error rate: la frazione di elementi che **non** appartengono alla classe dominante
- Gini index
- Entropia

$$G(S) = 1 - \sum_{j=1}^k p_j^2, \quad \text{Gini-Split } (S \Rightarrow S_1 \dots S_r) \Rightarrow \sum_{i=1}^r \frac{|S_i|}{|S|} G(S_i)$$

Suddivisione del data set in r sottoinsiemi
per uno split a r vie $\rightarrow$ *si sceglie lo split con
Error rate, Gini index o Entropia più bassa*

$$E(S) = - \sum_{j=1}^k p_j \log_2(p_j), \quad \text{Entropy-Split } (S \Rightarrow S_1 \dots S_r) \Rightarrow \sum_{i=1}^r \frac{|S_i|}{|S|} E(S_i)$$

Decision Tree

- In genere i nodi vicini ai nodi foglia lavorano su pochi dati e sono pronti al rumore nell'applicare lo split
 - Tendenza ad esibire overfitting
 - Si utilizzano tecniche di stop che prediligono alberi poco profondi
- Si utilizzano tecniche di pruning dei nodi foglia che vanno in overfit
 - Si valida l'eventuale incremento dell'accuracy su un validation set espunto dai dati di addestramento

Classificatori probabilistici

- L'appartenenza di un campione ad una classe è espressa in termini di una probabilità
- (Naive) Bayes Classifier
 - Si usa il Teorema di Bayes per stimare la probabilità di classe dall'evidenza delle feature *che si suppongono categoriche*
 - Il campione $\mathbf{x}$ è espresso come d variabili statistiche: $\{x_1 = a_1, x_2 = a_2, \dots, x_d = a_d\}$

Classificatori probabilistici

- L'appartenenza di un campione ad una classe è espressa in termini di una probabilità
- (Naive) Bayes Classifier
 - Il modello è una mistura di distribuzioni di natura generativa analogamente al clustering
 - Quindi si addestra con EM

Classificatori probabilistici

- L'appartenenza di un campione ad una classe è espressa in termini di una probabilità
- (Naive) Bayes Classifier
 - L'ipotesi «naive» è quella dell'indipendenza statistica delle feature che consente il calcolo della probabilità di un campione come produttoria delle probabilità delle singole feature

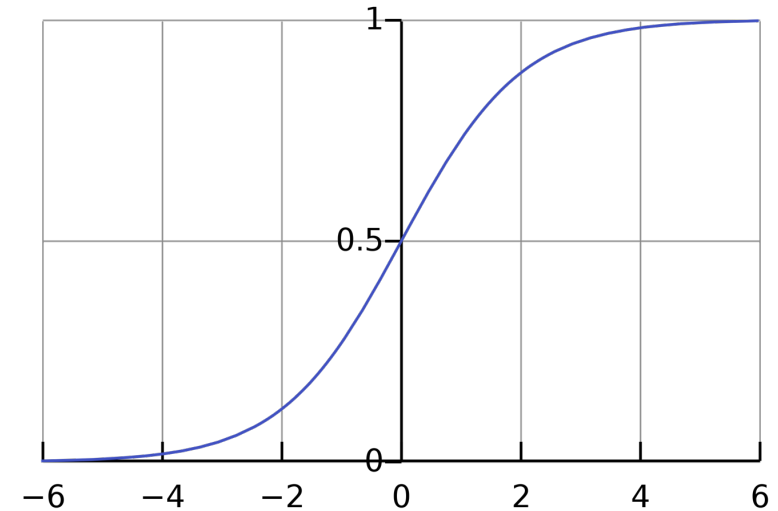
$$P(x_1 = a_1, \dots, x_d = a_d | C = c) = \prod_{j=1}^d P(x_j = a_j | C = c)$$

Classificatori probabilistici

- L'appartenenza di un campione ad una classe è espressa in termini di una probabilità
- Regressione logistica
 - E' un modello discriminativo tra due classi
 - La probabilità della variabile di classe è una distribuzione di Bernoulli parametrizzata in termini delle feature
 - Si cercherà così l'iperpiano di separazione tra le due classi

Regressione logistica

- La regressione logistica prende il nome dalla funzione di discriminare della probabilità di appartenenza del campione ad una di due classi individuate come $\{+1, -1\}$.
- Il modello dipende da $d + 1$ parametri θ_i che individuano la separazione tra le classi lungo ogni dimensione del campione



$$\sigma(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{1 + e^x}$$

Funzione logistica o sigmoide

$$P(C = +1|\bar{X}) = \frac{1}{1 + e^{-(\theta_0 + \sum_{i=1}^d \theta_i x_i)}}$$

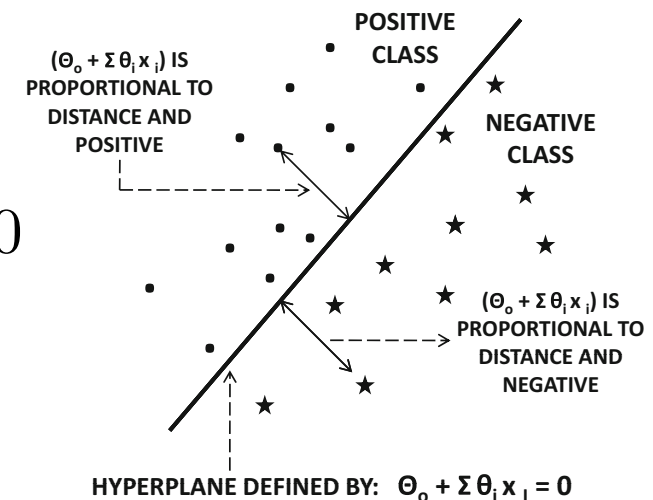
$$P(C = -1|\bar{X}) = \frac{1}{1 + e^{(\theta_0 + \sum_{i=1}^d \theta_i x_i)}}$$

Regressione logistica

- La regressione logistica è un «modello lineare» perché individua l'iperpiano di separazione tra le due classi in termini del luogo dei punti equiprobabili rispetto all'appartenenza ad una delle due classi

$$P(C = +1) \equiv P(C = -1) = 0.5 \rightarrow \theta_0 + \sum_{i=1}^d \theta_i x_i = 0$$

- Il modello si addestra massimizzando la log-likelihood del data set così definita



Instance based learning

- L'idea su cui si basano questi classificatori è

«Campioni simili hanno etichetta di classe simile»

- Sono dei «lazy learners»:
 - Il modello addestrato viene costruito solo *alla fine* dell'osservazione di tutti i dati

Instance based learning

- Classificatore k-NN
 - Il classificatore osserva i k elementi più vicini al campione ed attribuisce la classe dominante tra questi
 - Si può pensare di pesare il contributo dei vicini rispetto alla loro distanza δ dal campione, per esempio $f(\delta) = e^{-\frac{\delta^2}{t^2}}$
 - k e t sono iperparametri e la loro scelta è cruciale

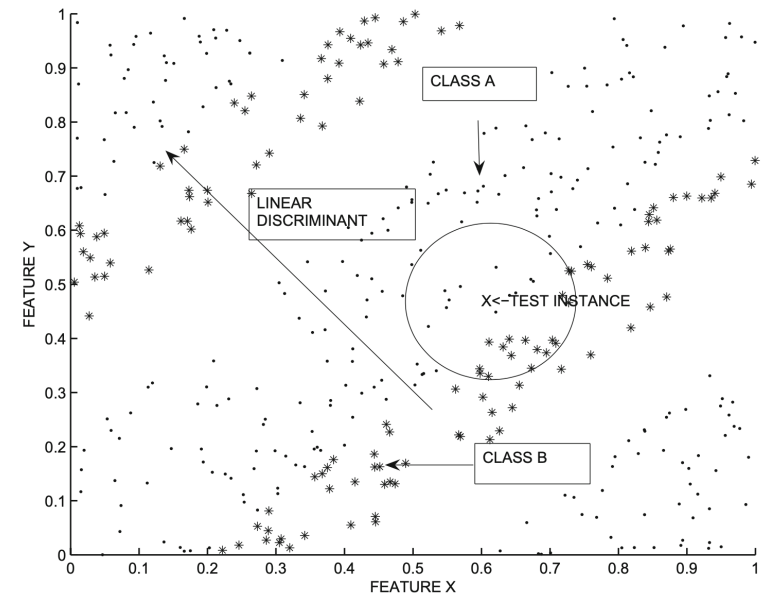
Instance based learning

- Classificatore k-NN
 - In generale si userà una distanza nella forma:

$$\text{Dist}(\bar{X}, \bar{Y}) = \sqrt{(\bar{X} - \bar{Y})A(\bar{X} - \bar{Y})^T}$$

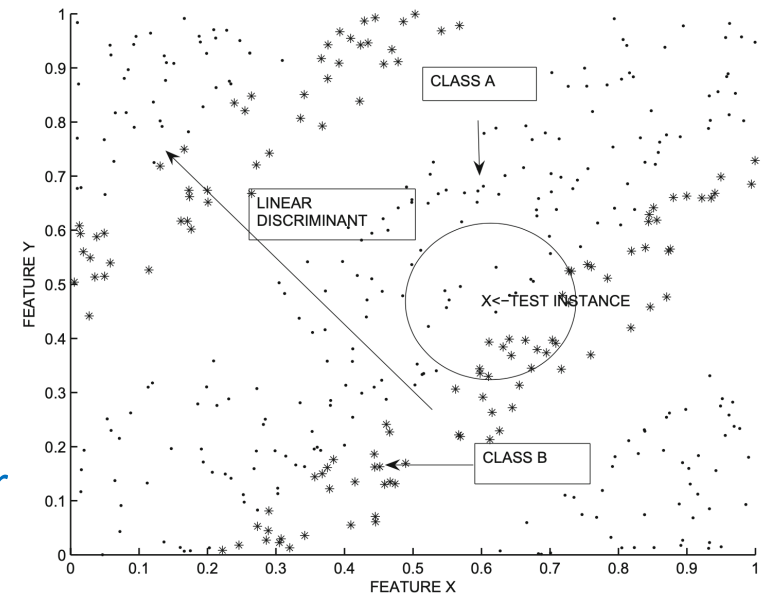
- $A = \Sigma^{-1}$ implica la distanza di Mahalanobis

- *Ma è la scelta migliore?*



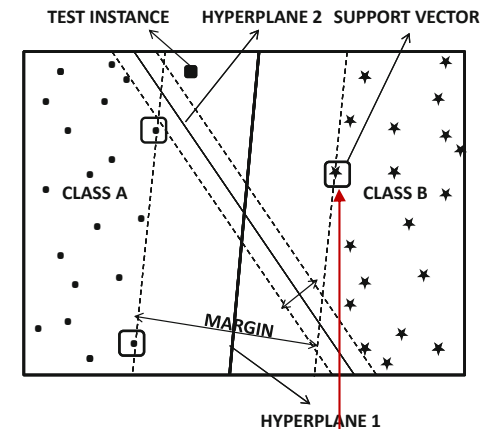
Instance based learning

- Classificatore k-NN
 - Si possono proiettare le distanze lungo le direzioni di massima variazione tra le diverse classi
 - Si ottiene la Fisher LDA se A viene posta uguale alla matrice che esprime il rapporto *intra / inter* nella funzione obiettivo di quel modello



Support Vector Machines

- Classificatore *lineare* per problemi di *classificazione binaria*
- Il modello ricerca l'*iperpiano a massimo margine* che separa le due classi
 - *Massimo margine*: la massima distanza tra due iperpiani paralleli che contengono alcune istanze di entrambe le classi dette support vector



Support Vector Machines

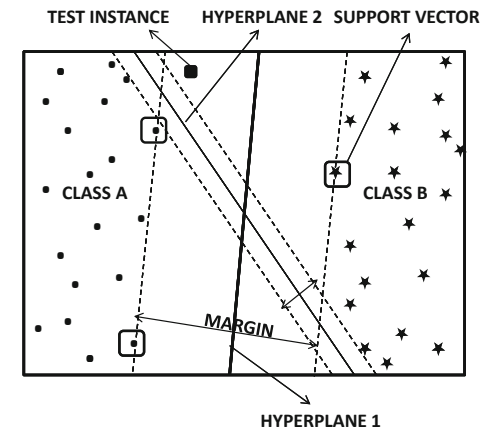
Le equazioni dei due iperpiani che limitano il margine si possono scrivere in termini dei coefficienti W e dell'intercetta b :

$$\overline{W} \cdot \overline{X}_i + b \geq 0 \quad \forall i : y_i = +1$$

$$\overline{W} \cdot \overline{X}_i + b \leq 0 \quad \forall i : y_i = -1$$

Con un apposito scaling di $\overline{W}$ e b :

$$\begin{cases} \overline{W} \cdot \overline{X}_i + b \geq +1 \forall i : y_i = +1 \\ \overline{W} \cdot \overline{X}_i + b \leq -1 \forall i : y_i = -1 \end{cases} \Rightarrow y_i(\overline{W} \cdot \overline{X}_i + b) \geq +1, \forall i$$



Support Vector Machines

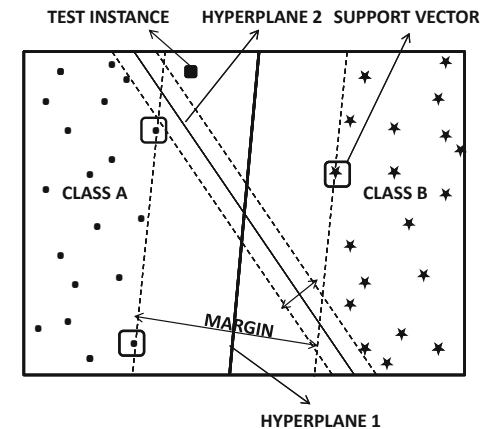
Un po' di ripasso di algebra.

Calcoliamo la distanza $d(\pi_1, \pi_2)$ tra due piani paralleli
 $\pi_1 = ax + by + cz + h$ e $\pi_2 = ax + by + cz + k$

La normale comune ai due piani è data dai coefficienti direttori: $\mathbf{n} = \{a, b, c\}$

$d(\pi_1, \pi_2) \equiv d(P, \pi_2)$, $P \in \pi_1$ posto lungo la direzione di $\mathbf{n}$

$$d(P, \pi_2) = |ax + by + cz + k| / ||\mathbf{n}||.$$

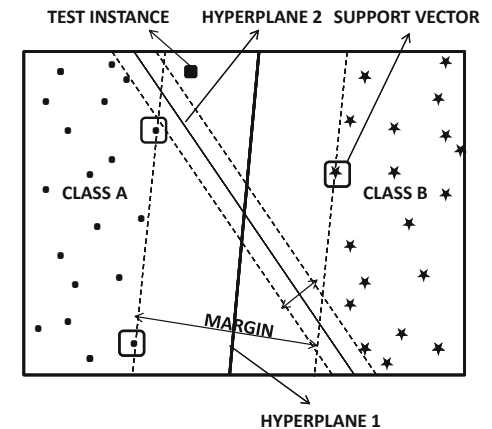


Support Vector Machines

In relazione al margine si deduce che vogliamo massimizzare la seguente funzione obiettivo

distanza iperpiani : $2/\|\bar{W}\|$

funzione obiettivo : $O = \|\bar{W}\|^2/2$



Si costruisce un funzionale da massimizzare con il metodo dei moltiplicatori di Lagrange e poi si costruisce una *funzione duale da minimizzare*

Support Vector Machines

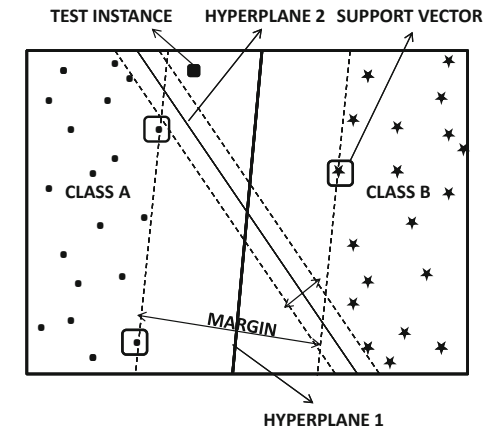
La funzione obiettivo risultante è:

$$L_D = \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j \overline{X_i} \cdot \overline{X_j}$$

L'addestramento avviene tramite discesa lungo il gradiente

$$\frac{\partial L_D}{\partial \lambda_i} = 1 - y_i \sum_{j=1}^n y_j \lambda_j \overline{X_i} \cdot \overline{X_j}$$

$$(\lambda_1 \dots \lambda_n) \leftarrow (\lambda_1 \dots \lambda_n) + \alpha \left(\frac{\partial L_D}{\partial \lambda_1} \dots \frac{\partial L_D}{\partial \lambda_n} \right)$$

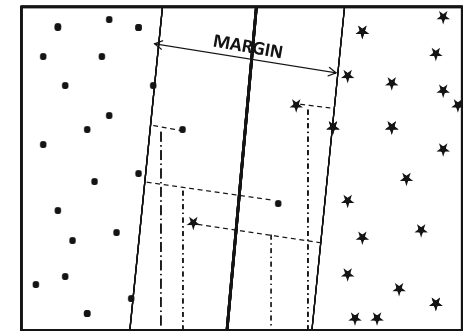


λ_i diversi da zero solo per i support vectors

Support Vector Machines

Esistono delle situazioni in cui alcuni punti violano leggermente la condizione di lineare separabilità

Si modifica la funzione obiettivo ammettendo delle *violazioni con penalità* sulla funzione obiettivo



MARGIN VIOLATION WITH PENALTY-BASED SLACK VARIABLES

$$\begin{cases} \overline{W} \cdot \overline{X}_i + b \geq +1 - \xi_i & \forall i : y_i = +1 \\ \overline{W} \cdot \overline{X}_i + b \leq -1 + \xi_i & \forall i : y_i = -1 \end{cases}, \quad \xi_i \geq 0 \quad \forall i$$

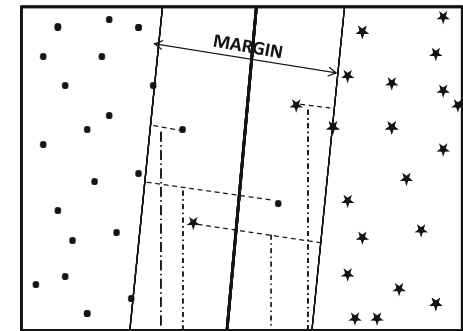
Slack variables

Support Vector Machines

Si modifica la funzione obiettivo ammettendo delle *violazioni con penalità* sulla funzione obiettivo

funzione obiettivo :

$$O = \frac{\|\bar{W}\|^2}{2} + C \sum_{i=1}^n \xi_i$$



MARGIN VIOLATION WITH PENALTY-BASED SLACK VARIABLES

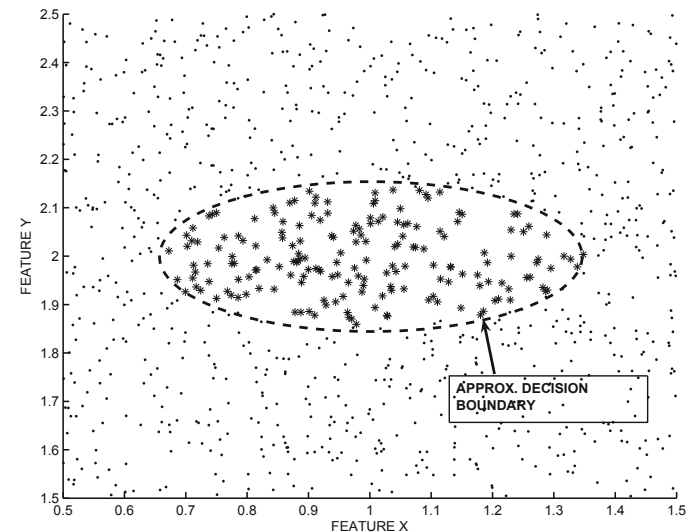
Il nuovo funzionale da minimizzare tiene conto anche dei punti che violano il margine con parametri propri β_i oltre ai parametri λ_i dei support vectors

Support Vector Machines

La Support Vector Machine può essere utilizzata per problemi non lineari con il *kernel trick*

$$L_D = \sum_{i=1}^n \lambda_i - \frac{1}{2} \cdot \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j K(\overline{X}_i, \overline{X}_j)$$

Function	Form
Gaussian radial basis kernel	$K(\overline{X}_i, \overline{X}_j) = e^{- \overline{X}_i - \overline{X}_j ^2 / 2\sigma^2}$
Polynomial kernel	$K(\overline{X}_i, \overline{X}_j) = (\overline{X}_i \cdot \overline{X}_j + c)^h$
Sigmoid kernel	$K(\overline{X}_i, \overline{X}_j) = \tanh(\kappa \overline{X}_i \cdot \overline{X}_j - \delta)$



Classificazione multiclasse

- Molti algoritmi di classificazione possono lavorare nativamente per più classi
 - Decision tree
 - Classificatori (Naive) Bayes
- La regressione logistica e la SVM sono concepite come classificatori binari
- Si possono definire strategie per estendere il loro comportamento alla gestione di problemi multiclasse

Classificazione multiclasse

- Strategia *one-against-rest*:
 - Si addestrano k classificatori per cui l' i -esimo modello binario predice come classe positiva l'appartenenza alla classe C_i
 - Nel test, se il modello i -esimo predice correttamente la classe C_i questa ottiene un voto
 - Altrimenti ottengono un voto tutte le altre classi
 - Il classificatore fornirà come etichetta di classe quella che ha ottenuto la maggioranza dei voti

Classificazione multiclasse

- Strategia *one-against-one*:
 - Si addestrano $\binom{k}{2}$ classificatori per tutte le coppie possibili di classi
 - Nel test, il generico modello $C_i\text{-against-}C_j$ attribuisce un voto alla classe correttamente predetta
 - Il classificatore fornirà come etichetta di classe quella che ha ottenuto la maggioranza dei voti
 - Molti più classificatori, ma training set più piccoli per cui risulta più conveniente dal punto di vista computazionale

Rare class learning

- La classificazione di un data set molto *sbilanciato* ovvero con dei campioni appartenenti ad una *classe rara* è interessante per diversi aspetti:
 - Nel dominio del problema, la classe rara è quella *interessante* ed un errore di classificazione nella classe rara è *molto dannoso*
 - I classificatori necessitano di tecniche di pesatura e/o campionamento dei dati ad hoc per gestire lo sbilanciamento

Rare class learning

- Pesatura dei campioni
 - Ogni campione ha un peso legato al costo della sua errata classificazione
→ privilegia le classi rare
 - Decision tree: il peso si incorpora nel criterio di split come Gini index o Entropia
 - Naive Bayes: si pesa la stima dei prior e delle probabilità delle feature



Università
degli Studi
di Palermo



Rare class learning

- Pesatura dei campioni
 - Ogni campione ha un peso legato al costo della sua errata classificazione
→ privilegia le classi rare
 - Support Vector Machines (con slack variables): le penalità di violazione del margine sono pesate per le diverse classi
 - k-NN: i voti rispetto alle singole classi dei k vicini vengono pesati

Rare class learning

- Campionamento
 - Si *sovracampiona* la classe rara ovvero si *campionano* le classi abbondanti
 - Campionamento con rimpiazzo o senza rimpiazzo
 - Il sovracampionamento può essere dannoso perché è ovviamente una tecnica con rimpiazzo e quindi genera più copie dello stesso campione
→ porta ad overfitting

Rare class learning

- Campionamento
 - Sovracampionamento sintetico – SMOTE:
 - Si selezionano i k vicini di un campione raro, appartenenti alla classe minoritaria
 - Si sceglie una frazione casuale di questi, a seconda del livello richiesto di sovracampionamento
 - Si genera un campione sintetico in un punto casuale lungo la linea congiungente il campione con ognuno dei vicini selezionati

Regressione

- Regressione lineare

- Si consideri un data set D di n vettori in $\mathbb{R}^d$ $D = \begin{bmatrix} \bar{X}_1 \\ \vdots \\ \bar{X}_n \end{bmatrix}$, $(n \times d)$ e siano definite n corrispondenze $X_i \rightarrow y_i, i = 1, \dots, n$

- Determinare un vettore di coefficienti $W = \{w_1, \dots, w_d\}$ che consenta di approssimare l'insieme delle corrispondenze $X_i \rightarrow y_i, i = 1, \dots, n$, come:

$$y_i \approx \bar{W} \cdot \bar{X}_i \quad \forall i \in \{1 \dots n\}$$

Regressione

- Regressione lineare
 - Si minimizza una funzione obiettivo definita come la somma degli errori quadratici (Sum of Squared Errors – SSE):

$$O = \sum_{i=1}^n (\bar{W} \cdot \bar{X}_i - y_i)^2 = \|D\bar{W}^T - \bar{y}\|^2$$

- Si parlerà di Least Squares Regression

$$\bar{W}^T = (D^T D)^{-1} D^T \bar{y}$$

Regressione

- Regressione lineare:

- Ridge regression: si utilizza una regolarizzazione L_2

$$O = \|D\bar{W}^T - \bar{y}\|^2 + \lambda \|\bar{W}\|^2$$

$$\bar{W}^T = (D^T D + \lambda I)^{-1} D^T \bar{y}$$

- La matrice $D^T D + \lambda I$ è semidefinita positiva e quindi sempre invertibile
 - la ridge regression riduce la varianza della stima rispetto alla least squares

Regressione

- Regressione lineare

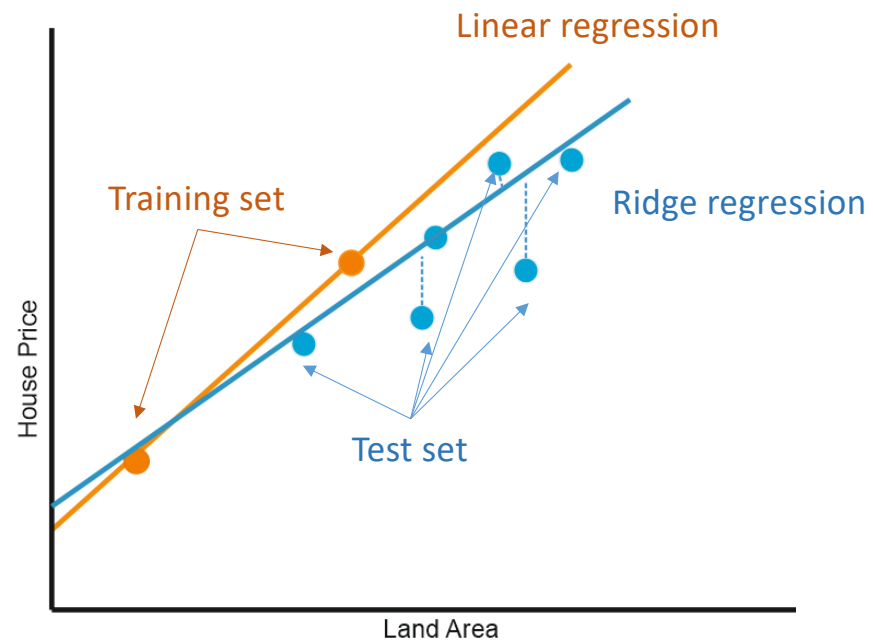
- Lasso regression: si utilizza una regolarizzazione L_1

$$O = \left\| D\bar{W}^T - \bar{y} \right\|^2 + \lambda \left\| \bar{W} \right\|$$

- La soluzione è iterativa e ammette, tendenzialmente, il caso $W = 0$
 - La regressione Lasso, in genere, predilige un vettore dei coefficienti sparso
 - Indicata nel caso di presenza di molte feature irrilevanti per il problema, le quali vedranno azzerarsi i rispettivi coefficienti

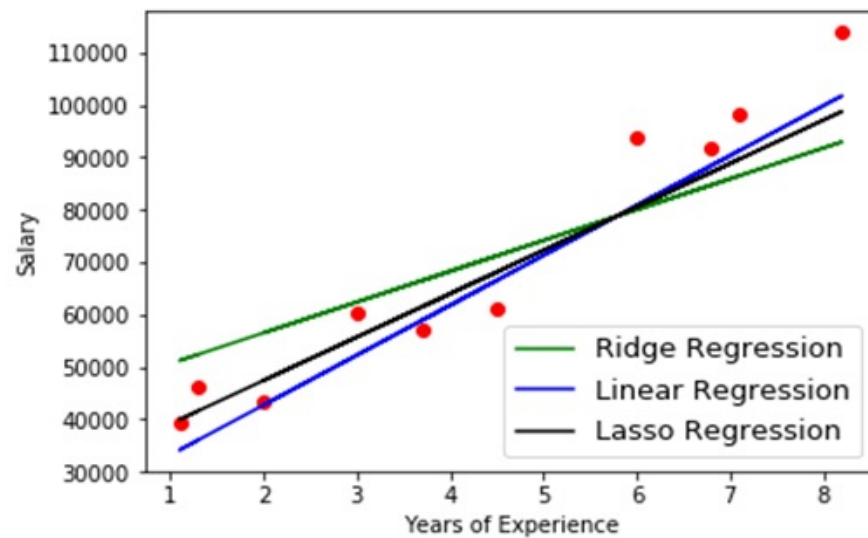
Regressione

- Effetto della regolarizzazione

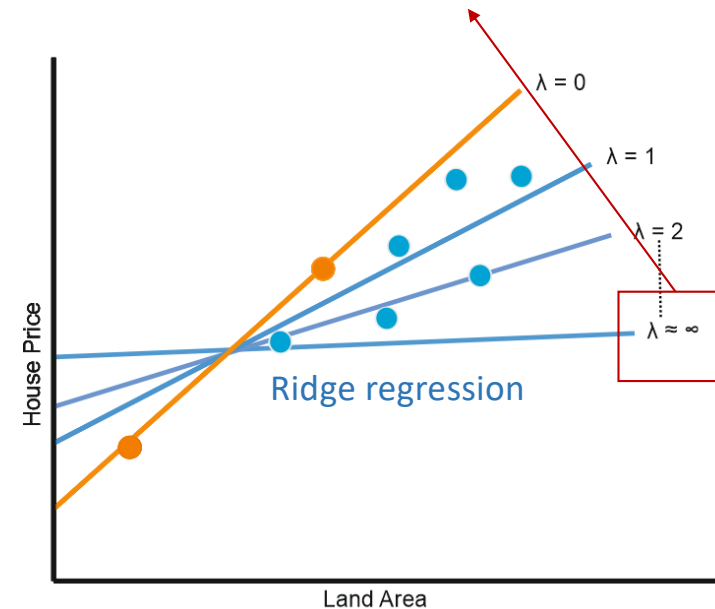


Regressione

- Ridge vs Lasso



La Ridge regression azzerava il contributo di una feature solo asintoticamente ($\lambda \rightarrow \infty$) mentre la Lasso regression può eliminare esattamente la feature (pendenza nulla per λ molto grande, ma non ∞)



Regressione

- Principal Component Regression

- Utilizzato per ridurre la dimensionalità del data set rispetto alle feature poco interessanti o rumorose

- Si applica la PCA a D ottenendo una nuova matrice dei dati $R = \begin{bmatrix} \bar{Z}_1 \\ \vdots \\ \bar{Z}_n \end{bmatrix}, (n \times k)$
proiettando D sulle k direzioni principali

- Si risolve, quindi il problema

$$y_i \approx \bar{W} \cdot \bar{Z}_i \quad \forall i \in \{1 \dots n\}, \quad \bar{W} = (R^T R)^{-1} R^T \bar{y}$$

Regressione

- Generalized Linear Models

- Modello generalizzato della regressione lineare in cui si stima una *funzione di media* $f(\bar{W} \cdot \bar{X}_i)$, $i = 1, \dots, n$ della distribuzione di probabilità delle etichette numeriche y_i
- Mantiene le caratteristiche della regressione lineare, ma si estende a diverse tipologie di dati
- Il problema da risolvere è quindi:

$y_i \sim$ Probability distribution with mean $f(\bar{W} \cdot \bar{X}_i) \quad \forall i \in \{1 \dots n\}$

Regressione non lineare

- Estensione della regressione ad una funzione non lineare del campione:

$$y = \sum_{i=1}^m w_i h_i(\bar{X})$$

- Il caso tipico è quello della regressione polinomiale in cui $h_i(\bar{X}) = \bar{X}^i$
- Cattura relazioni funzionali complesse tra i punti del data set

Regressione non lineare

- Kernel ridge regression
 - Si usa il kernel trick per modellare la relazione tra i dati e le etichette numeriche
 - La soluzione della ridge regression calcola la matrice $\bar{W}^T$ per cui la predizione sui dati di test $\bar{Z}$ è $F(\bar{Z}) = \bar{Z}\bar{W}^T$
 - Il kernel K quindi andrà sostituito sia nei prodotti scalari tra i dati sia in quelli *tra i vettori di test e i dati*

Regressione

- Alberi di regressione
 - Modella le relazioni non lineari tra i dati come un insieme di *approssimazioni lineari locali* nei nodi foglia dell'albero
 - Lo split per variabili categoriche è quello tipico di un decision tree (Gini index, entropia) e così pure le tecniche di pruning
 - Per dati numerici si minimizza il valore di SSE su tutti i nodi risultanti dallo split

Regressione

- Alberi di regressione
 - Modella le relazioni non lineari tra i dati come un insieme di *approssimazioni lineari locali* nei nodi foglia dell'albero
 - Il modello richiederebbe una regressione ad ogni nodo.
 - In alternativa si può controllare la varianza dei dati ad ogni split e poi fare la regressione nei nodi foglia *dopo* aver costruito l'albero



Università
degli Studi
di Palermo

dipartimento
di ingegneria
unipa



Regressione

- Valutazione della regressione
 - Si usa la statistica R^2 o la sua variante per d elevato:

$$R^2 = 1 - \frac{SSE}{SST} \quad R^2 = 1 - \frac{(n - d) SSE}{(n - 1) SST}$$

- In cui

$$SSE = \sum_{i=1}^n (y_i - g(\bar{X}_i))^2 \quad SST = \sum_{i=1}^n \left(y_i - \sum_{j=1}^n \frac{y_j}{n} \right)^2$$

Metodi di ensemble

- I metodi di ensemble combinano le predizioni di un insieme di algoritmi (base learners) sul data set
- Sfruttano le caratteristiche peculiari di ciascun algoritmo rispetto ai dati per incrementare l'accuratezza

Algorithm *EnsembleClassify*(Training Data Set: $\mathcal{D}$
Base Algorithms: $\mathcal{A}_1 \dots \mathcal{A}_r$, Test Instances: $\mathcal{T}$)
begin
 $j = 1$;
 repeat
 Select an algorithm $\mathcal{Q}_j$ from $\mathcal{A}_1 \dots \mathcal{A}_r$;
 Create a new training data set $f_j(\mathcal{D})$ from $\mathcal{D}$;
 Apply $\mathcal{Q}_j$ to $f_j(\mathcal{D})$ to learn model $\mathcal{M}_j$;
 $j = j + 1$;
 until(termination);
 report labels of each $T \in \mathcal{T}$ based on combination of
 predictions from all learned models $\mathcal{M}_j$;
end

Metodi di ensemble

- I metodi di ensemble orientati ai dati
 - $\mathcal{A}_1, \dots, \mathcal{A}_r$ sono istanze dello stesso modello
 - $f_j(\mathcal{D})$ è differente per ciascun modello
 - Campionamento
 - Manipolazione delle feature o delle etichette
 - Focalizzazione su porzioni di dati in cui la classificazione non è corretta

Algorithm *EnsembleClassify*(Training Data Set: $\mathcal{D}$

Base Algorithms: $\mathcal{A}_1 \dots \mathcal{A}_r$, Test Instances: $\mathcal{T}$)

begin

$j = 1$;

repeat

 Select an algorithm $\mathcal{Q}_j$ from $\mathcal{A}_1 \dots \mathcal{A}_r$;

 Create a new training data set $f_j(\mathcal{D})$ from $\mathcal{D}$;

 Apply $\mathcal{Q}_j$ to $f_j(\mathcal{D})$ to learn model $\mathcal{M}_j$;

$j = j + 1$;

until(termination);

report labels of each $T \in \mathcal{T}$ based on combination of predictions from all learned models $\mathcal{M}_j$;

end

Metodi di ensemble

- I metodi di ensemble orientati ai modelli
 - $\mathcal{A}_1, \dots, \mathcal{A}_r$ sono istanze di modelli differenti
 - $f_j(\mathcal{D}) \equiv \mathcal{D}$
 - Differenti modelli esibiscono performance migliori in diverse porzioni del data set

Algorithm *EnsembleClassify*(Training Data Set: $\mathcal{D}$

Base Algorithms: $\mathcal{A}_1 \dots \mathcal{A}_r$, Test Instances: $\mathcal{T}$)

begin

$j = 1$;

repeat

 Select an algorithm $\mathcal{Q}_j$ from $\mathcal{A}_1 \dots \mathcal{A}_r$;

 Create a new training data set $f_j(\mathcal{D})$ from $\mathcal{D}$;

 Apply $\mathcal{Q}_j$ to $f_j(\mathcal{D})$ to learn model $\mathcal{M}_j$;

$j = j + 1$;

until(termination);

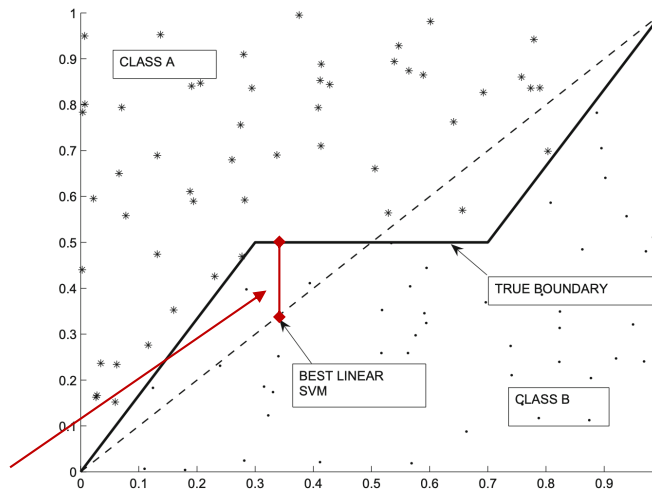
report labels of each $T \in \mathcal{T}$ based on combination of predictions from all learned models $\mathcal{M}_j$;

end

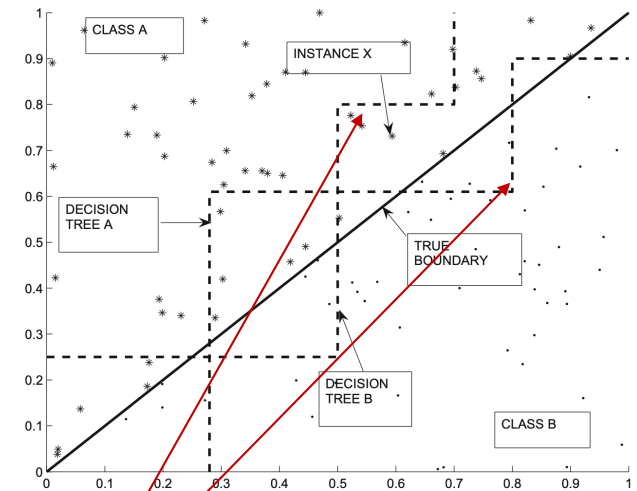
Metodi di ensemble

- I metodi di ensemble migliorano *sia il bias* *sia la varianza della stima* del classificatore

Un singolo classificatore esibisce un proprio *bias* rispetto alla reale distribuzione delle classi *anche per un numero elevato di dati di addestramento*



(a) bias



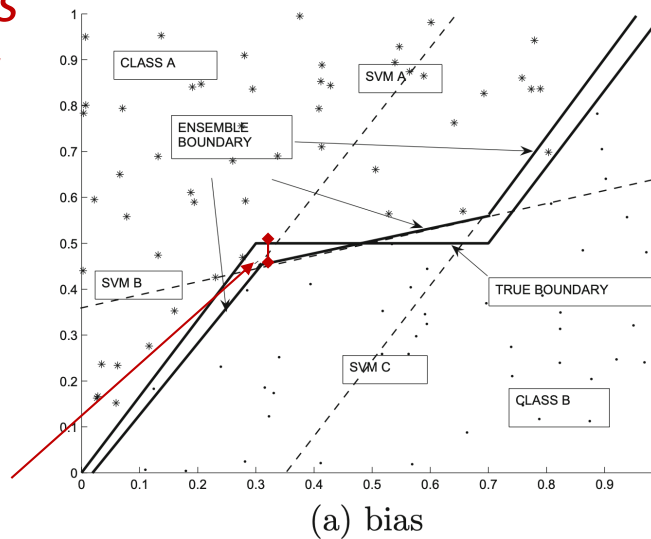
(b) variance

Un singolo classificatore ha *varianza* insita nella stima perché al variare dell'istanza del training set la predizione del medesimo campione può cambiare

Metodi di ensemble

- I metodi di ensemble migliorano *sia il bias* *sia la varianza della stima* del classificatore

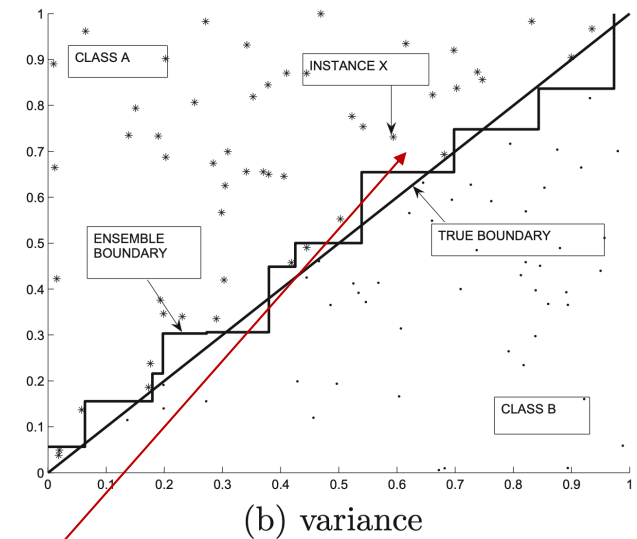
Un ensemble di classificatori approssima meglio un boundary complesso riducendo il bias



Tre decision tree indipendenti:

$$p_{tree,1} = p_{tree,2} = p_{tree,3} = p_{tree} = 80\%$$

$$p_{ensemble} = p_{tree,1} \times p_{tree,2} \times p_{tree,3} + \binom{3}{2} p_{tree} \times p_{tree} \times (1 - p_{tree}) = \mathbf{89.6\%}$$



All'aumentare del numero dei componenti dell'ensemble la stima riduce la propria varianza

Metodi di ensemble

- Bagging (Bootstrapped aggregating)
 - Tecnica per creare un ensemble di classificatori i.i.d. in modo tale da ridurre la varianza della stima da σ^2 a σ^2/k
 - Si generano k data set tramite campionamento con rimpiazzo: i nuovi data set contengono duplicati
 - Si può mostrare che la frazione di dati distinti in ogni data set $1 - (1 - 1/n)^n \approx 1 - 1/e$ per n elevato
 - Questa tecnica è detta Bootstrap: usata anche per calcolare l'Accuracy di un modello pesata su più istanze del data set «lievemente» diverse

Metodi di ensemble

- Bagging (Bootstrapped aggregating)
 - Si addestrano i k classificatori e la predizione è data dalla maggioranza o dalla media dei voti espressi da ciascuno su ogni campione
 - Riduce la varianza della stima, *ma non il bias*
 - Conviene scegliere dei modelli che tendano di per sé a ridurre il bias

Random Forests

- È un ensemble di decision tree addestrati con il bagging
- Il decision tree, soprattutto se fatto accrescere molto in profondità, riduce molto il bias anche se a forte scapito della varianza della classificazione
- È un modello ideale per il bagging

Random Forests

- I nodi più elevati della gerarchia rimangono sostanzialmente invariati nei diversi alberi
- I singoli alberi sono molto correlati e ciò limita la riduzione dell'errore di stima
- È necessario diversificare molto gli alberi durante la loro crescita per aumentare la capacità di predizione dell'ensemble
- Le Random Forests usano un *criterio random per lo split nei diversi alberi* dell'ensemble

Random Forests

- Random split selection
- Forest-RI (Random Input): Un sottoinsieme casuale di q feature viene estratto per lo split
- Forest-RC (Random Combination): un sottoinsieme L di feature viene estratto e si creano q combinazioni lineari che vengono usate come feature multivariate per lo split
 - Preferibile per bassa dimensionalità dei dati per cui q diventa molto piccolo

Boosting

- Tecnica di ensemble iterativa orientata a ridurre il bias
- Lo stesso learner viene riaddestrato più volte sul data set in cui ogni campione è pesato *con un peso maggiore se è stato mis-classificato all'iterata precedente*
- Il classificatore all'iterata t viene guidato dagli errori commessi dalle iterate precedenti

Boosting

- AdaBoost

Predizione:

somma pesata delle predizioni
ad ogni iterata t tramite il
coefficiente α_t

Si può ridurre la varianza se si
ricampiona il data set ad ogni
iterazione

Poco indicato per dati
rumorosi

Il coefficiente di peso cresce logicamente
al decrescere dell'error rate

Algorithm *AdaBoost*(Data Set: $\mathcal{D}$, Base Classifier: $\mathcal{A}$, Maximum Rounds: T)
begin

$t = 0$;

for each i initialize $W_1(i) = 1/n$;

repeat

$t = t + 1$;

Determine weighted error rate ϵ_t on $\mathcal{D}$ when base algorithm $\mathcal{A}$
is applied to weighted data set with weights $W_t(\cdot)$;

$\alpha_t = \frac{1}{2} \log_e((1 - \epsilon_t)/\epsilon_t)$;

for each misclassified $\bar{X}_i \in \mathcal{D}$ **do** $W_{t+1}(i) = W_t(i)e^{\alpha_t}$;

else (correctly classified instance) **do** $W_{t+1}(i) = W_t(i)e^{-\alpha_t}$;

for each instance $\bar{X}_i$ **do** normalize $W_{t+1}(i) = W_{t+1}(i)/[\sum_{j=1}^n W_{t+1}(j)]$;

until $((t \geq T) \text{ OR } (\epsilon_t = 0) \text{ OR } (\epsilon_t \geq 0.5))$;

Use ensemble components with weights α_t for test instance classification;

end

Frazione dei campioni
mis-classificati

Peso crescente

Peso decrescente

Classificazione
perfetta

Performance peggiore del
classificatore casuale

Boosting

- XGBoost (eXtreme Gradient Boosting)
 - Algoritmo molto recente di ensemble per compiti di classificazione e regressione, basato sul concetto di *gradient boosting*
 - L'idea di base è che: *«il boosting può essere interpretato come la minimizzazione di un'apposita funzione di costo nello spazio funzionale dei modelli»*



Università
degli Studi
di Palermo



Boosting

- XGBoost (eXtreme Gradient Boosting)

- Gradient boosting:

$$\hat{y} = F(x)$$

- Si consideri il problema di classificazione:

$$L_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^n (y_i - F(x_i))^2$$

- Lo schema di boosting aggiorna il modello come segue:

$$F_{m+1}(x_i) = F_m(x_i) + h_m(x_i) = y_i$$

$$h_m(x_i) = y_i - F_m(x_i)$$

*Learner addestrato sui
dati al passo m*

Residui $\rightarrow$ gradienti di L_{MSE}

Valutazione della bontà della classificazione

La più semplice misura è l'accuracy ovvero la percentuale di elementi predetti correttamente in tutte le classi.

Fornisce una falsa informazione se le classi sono fortemente sbilanciate.

$$ACC = \frac{1}{N} \sum_{k=1}^C \sum_{x \in C_k} I(C(x) \equiv C_k), \quad I = \begin{cases} 0, & C(x) \neq C_k \\ 1, & C(x) = C_k \end{cases}$$

$$bACC = \frac{1}{C} \sum_{k=1}^C \frac{1}{|C_k|} \sum_{x \in C_k} I(C(x) \equiv C_k)$$

Valutazione della bontà della classificazione

Predizione → Reference ↓	C _i	Altre classi
C _i	TP _i	FN _i
Altre classi	FP _i	TN _i

$$Acc_i = \frac{TP_i + TN_i}{TP_i + TN_i + FP_i + FN_i}$$

$$P_i = \frac{TP_i}{TP_i + FP_i}$$

$$R_i = \frac{TP_i}{TP_i + FN_i}$$

$$F_1 = 2 \frac{P_i \cdot R_i}{P_i + R_i}$$

$$P_{\text{micro}} = \frac{\sum_{i=1}^C TP_i}{\sum_{i=1}^C TP_i + FP_i}$$

$$R_{\text{micro}} = \frac{\sum_{i=1}^C TP_i}{\sum_{i=1}^C TP_i + FN_i}$$

$$F_{1\text{micro}} = 2 \frac{P_{\text{micro}} \cdot R_{\text{micro}}}{P_{\text{micro}} + R_{\text{micro}}}$$

$$P_{\text{macro}} = \frac{1}{C} \sum_{i=1}^C \frac{TP_i}{TP_i + FP_i} = \frac{\sum_{i=1}^C P_i}{C}$$

$$R_{\text{macro}} = \frac{1}{C} \sum_{i=1}^C \frac{TP_i}{TP_i + FN_i} = \frac{\sum_{i=1}^C R_i}{C}$$

$$F_{1\text{macro}} = 2 \frac{P_{\text{macro}} \cdot R_{\text{macro}}}{P_{\text{macro}} + R_{\text{macro}}}$$



Valutazione della bontà della classificazione

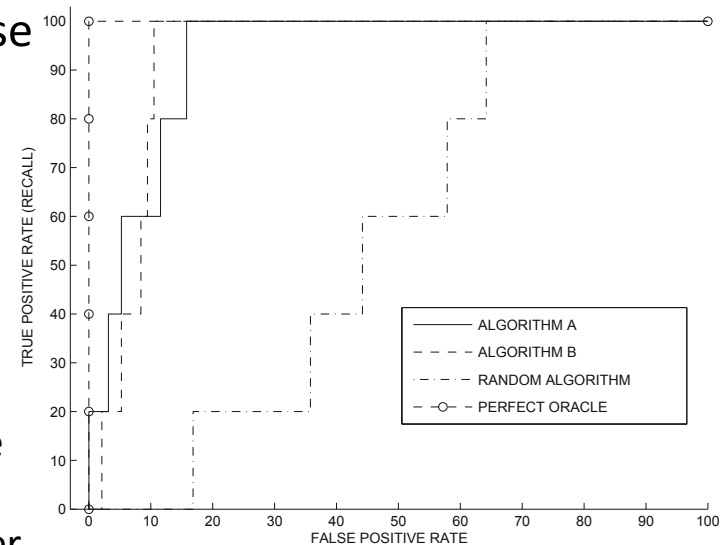
La Receiver Operating Curve (ROC) è tracciata misurando il False Positive Rate e il True Positive Rate al variare di una soglia definita sullo score della classe positiva.

$$ROC(t) \propto FPR(t) \text{ vs. } TPR(t)$$

$$TPR(t) \equiv Recall(t), FPR(t) = \frac{FP(t)}{FP(t) + TN(t)}$$

AUC, Area Under the Curve, tende a 1 per buona classificazione

Per i problemi multi-classe si *devono* tracciare più curve, una per singola classe in modalità one-vs-rest, e calcolare la relativa AUC.



(a) ROC